

Java-Kompass mit GPX-Track

im Studiengang

Informatik

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Moritz Pfitzenmaier, Marius Müllmaier, Daniel Bensch

12.06.2026

Bearbeitungszeitraum: April 2026 - Juni 2026

Matrikelnummer: 8829906, 1341874, 5133713

Abstract

Richtung und Entfernung zu einem Ziel zu bestimmen, gehört zu den Grundaufgaben der mobilen Navigation. Die iOS-Anwendung *Kompass Professional* zeigt diese Peilung anschaulich, verzahnt ihre Berechnungslogik aber fest mit der Benutzeroberfläche und der Sensorhardware. Die vorliegende Arbeit löst diese Logik heraus und überführt sie in eine eigenständige Java-Bibliothek ohne Benutzeroberfläche.

Den Kern bildet die Berechnung von Azimut, Distanz und Himmelsrichtung aus WGS84-Koordinaten. Positions- und Kursdaten erhält die Bibliothek von einem Host-System, eigene Sensoren liest sie nicht. Über einen steuerbaren Session-Lebenszyklus lassen sich Tracks aufzeichnen, unterbrechen und beenden, wobei konfigurierbare Reduktionsverfahren die Datenmenge begrenzen. Den aufgezeichneten Track exportiert sie im Format GPX 1.1, optional bindet sie den Dienst What3Words an nach dem hinzufügen eines API-keys.

Neben der Implementierung als Maven-Projekt enthält die Arbeit eine Spezifikation nach IEEE 830. Alle Anforderungen sind nach den SOPHIST-Regeln formuliert und auf konkrete Testfälle rückführbar. Hinzu kommen nicht-funktionale Anforderungen, ein objektorientiertes Analyse- und Entwurfsmodell sowie automatisierte Tests, deren Bezug zu den Anforderungen eine Traceability-Matrix festhält.

Inhaltsverzeichnis

Abstract	i
Inhaltsverzeichnis	ii
Abbildungsverzeichnis	iv
Tabellenverzeichnis	vi
1 Einleitung	1
1.1 Zweck	1
1.2 Einsatzbereich	2
1.2.1 Problemstellung und fachlicher Hintergrund	2
1.2.2 Funktionsumfang	2
1.3 Glossar	3
1.3.1 Fachbegriffe Navigation und Geografie	3
1.3.2 Begriffe zu Datenformaten und Protokollen	6
1.3.3 Begriffe zu Session und Systemverhalten	8
1.3.4 Begriffe zu Entwurfsmustern und Architektur	10
1.3.5 Begriffe zu Sicherheit	12
1.3.6 Begriffe zu Algorithmen	13
1.3.7 Begriffe zu Methodik und Qualitätssicherung	13
1.3.8 Begriffe zu Werkzeugen und Frameworks	14
1.4 Überblick	20
2 Allgemeine Beschreibung	21
2.1 Einbettung	21
2.1.1 Systemsicht und Systemgrenze	21
2.1.2 Schichtenarchitektur	21
2.1.3 Kommunikationsregeln	23
2.1.4 Physische Sicht	23
2.2 Funktionen	23
2.3 Benutzerprofile	25
2.3.1 Nutzercharakteristiken	26
2.3.2 Projektkontext (Stakeholder)	26
2.4 Einschränkungen	27
2.4.1 Im Scope	27
2.4.2 Außerhalb des Scopes	27
2.5 Annahmen und Abhängigkeiten	27
2.5.1 Annahmen	27
2.5.2 Abhängigkeiten	28
3 Spezifische Anforderungen	30

3.1 Funktionale Anforderungen	30
3.2 Nicht-funktionale Anforderungen	53
3.3 Externe Schnittstellen	55
3.3.1 Öffentliche Java-API	55
3.3.2 Fehlerverhalten	57
3.3.3 GPX 1.1-Ausgabeformat	58
3.4 Qualitäts-Anforderungen	59
3.4.1 Qualitätsmodell nach ISO/IEC 25010	59
3.4.2 Produktdaten (/LD.../)	60
3.4.3 Testfälle (Traceability)	62
3.4.4 FMEA	63
3.4.5 Whitebox-Sicht: Kontrollfluss und Entscheidungslogik	64
4 Anhang	66
4.1 Vollständige Traceability-Matrix	66
4.2 Objektorientiertes Analyse- und Entwurfsmodell (OOA/OOD)	67
4.2.1 OOA: Domänenmodell	67
4.2.2 OOD: Technischer Feinentwurf	67
4.2.3 Zustandsdiagramm: Session-Lebenszyklus	70
4.3 Struktur-, Architektur- und Verteilungssicht	71
4.4 Dynamische Modelle und Strukturierte Analyse	74
4.5 Haversine- und Azimutberechnung (Referenzalgorithmus)	78
4.6 Literatur- und Normenverweise	78

Abbildungsverzeichnis

Abbildung 1	Architekturschichten und ihr Kontrollfluss	22
Abbildung 2	Use-Case-Diagramm: Anwendungsfälle der öffentlichen API mit Primär- und Sekundärakteuren.	30
Abbildung 3	Aktivitätsdiagramm /LF010/	32
Abbildung 4	Aktivitätsdiagramm /LF020/	33
Abbildung 5	Aktivitätsdiagramm /LF030/	34
Abbildung 6	Aktivitätsdiagramm /LF040/	35
Abbildung 7	Aktivitätsdiagramm /LF050/	36
Abbildung 8	Aktivitätsdiagramm /LF060/	37
Abbildung 9	Aktivitätsdiagramm /LF070/	38
Abbildung 10	Aktivitätsdiagramm /LF080/	39
Abbildung 11	Aktivitätsdiagramm /LF090/	40
Abbildung 12	Aktivitätsdiagramm /LF100/	41
Abbildung 13	Kontextdiagramm (DFD Stufe 0): Datenflüsse zwischen Bibliothek und Umgebung.	55
Abbildung 14	Sequenzdiagramm <code>onPositionUpdate()</code> : Validierung, Aggregation und Listener-Ereignisse (/LF030/).	57
Abbildung 15	Sequenzdiagramm <code>complete()</code> : Export-Kette von der Aggregation bis zum <code>GpxResult</code> (/LF060/, /LF140/).	57
Abbildung 16	ER-Modell der Produktdaten (/LD010/ ... /LD060/) mit Entitäten und Kardinalitäten.	60
Abbildung 17	Kontrollflussgraph von <code>TrackAggregator.accept</code> (Whitebox-Sicht, $v(G) = 6$). .	64
Abbildung 18	Entscheidungstabelle der Fix-Verarbeitung (Bedingungen und Regeln R1–R5).	65
Abbildung 19	Entscheidungsbaum der Fix-Verarbeitung (Baumform der Entscheidungstabelle).	65
Abbildung 20	Klassendiagramm (OOD): Fassade, Value Objects, Domain Services, Strategy, Observer und Ports.	69
Abbildung 21	Objektdiagramm: Momentaufnahme einer aktiven Session (ACTIVE, zwei gespeicherte Punkte).	69
Abbildung 22	Zustandsdiagramm des Session-Lebenszyklus.	70
Abbildung 23	Zustandsdiagramm (detailliert) des Session-Lebenszyklus mit Guards und Effekten.	71
Abbildung 24	Paketdiagramm: Maven-Module und Java-Pakete mit ihren Abhängigkeiten.	71

Abbildung 25	Komponentendiagramm: Module mit bereitgestellten und benötigten Schnittstellen.	72
Abbildung 26	Kompositionsstrukturdiagramm: Innenleben von <code>DefaultBearingSession</code>	72
Abbildung 27	Verteilungsdiagramm: eingebetteter Betrieb mit optionalen externen Kanälen.	73
Abbildung 28	Architekturmuster Ports-and-Adapters (hexagonal).	73
Abbildung 29	Architekturmuster Pipes-and-Filter: die Optimierer-Pipeline.	74
Abbildung 30	Aktivitätsdiagramm: vollständiger Verlauf einer Peilungssession.	75
Abbildung 31	Interaktionsübersichtsdiagramm: Ablauf als Folge von Interaktionsverweisen.	76
Abbildung 32	Datenflussdiagramm (DFD Stufe 1): Teilprozesse und Datenspeicher.	77
Abbildung 33	Petri-Netz des Session-Lebenszyklus (Stellen, Transitionen, eine Marke). . . .	77

Tabellenverzeichnis

Tabelle 1	Externe Schnittstellen der Bibliothek und ihre Kommunikationsrichtungen.	21
Tabelle 2	Subsysteme und ihre Verantwortlichkeiten.	22
Tabelle 3	Erlaubte und verbotene Abhängigkeiten zwischen den Subsystemen.	23
Tabelle 4	Hauptfunktionsgruppen der Java-Peilungskomponente mit Verweis auf Anforderungen.	24
Tabelle 5	Entwurfsprinzipien und deren messbarer Nutzen.	25
Tabelle 6	Nutzergruppen der Bibliothek mit Erfahrung, Voraussetzungen und Erwartungen.	26
Tabelle 7	Stakeholder der Java-Peilungskomponente mit Erwartungen und Einfluss.	26
Tabelle 8	Übersicht der getroffenen Systemannahmen	27
Tabelle 9	Übersicht der Systemabhängigkeiten und externen Voraussetzungen	28
Tabelle 10	Funktionaler Anforderungskatalog /LF010/ ... /LF250/	30
Tabelle 11	Nicht-funktionale Anforderungen /LL010/ ... /LL080/.	54
Tabelle 12	Öffentliche API-Methoden der Bibliothek (<code>BearingSession</code> / <code>DefaultBearingSession</code>).	56
Tabelle 13	Geworfene Exceptions nach Auslöser (Vertrag der öffentlichen API).	58
Tabelle 14	Gewichtung der Qualitätsmerkmale nach ISO/IEC 25010 (Relevanzstufen je Unterkriterium).	59
Tabelle 15	Produktdaten /LD010/ ... /LD060/.	61
Tabelle 16	Testfälle mit Traceability zu /LF.../ und /LL.../.	62
Tabelle 17	FMEA: Fehlermodi, Folgen, Klassen und Gegenmaßnahmen.	63
Tabelle 18	Vollständige Traceability-Matrix	66
Tabelle 19	Fachliches Domänenmodell (OOA): Konzepte und ihre Beziehungen.	67
Tabelle 20	OOD-Klassenübersicht: Zentrale Entwurfsklassen und Schnittstellen.	67
Tabelle 21	Primäre Normreferenzen und technische Quellen des Projekts.	78

1 Einleitung

Dieses Kapitel klärt, welchen Zweck die Java-Peilungskomponente erfüllt, in welchem fachlichen Umfeld sie entsteht und wo ihre Grenzen liegen. Es benennt die Ziele der Bibliothek und die Kriterien, an denen sich ihr Erfolg messen lässt. Die technische Abgrenzung des Funktionsumfangs folgt in Kapitel 2.

1.1 Zweck

Ziel dieses Dokuments ist eine vollständige und prüfbare Spezifikation einer Java-Bibliothek ohne Benutzeroberfläche, welche die Peilung, die GPS-Track-Aufzeichnung und den GPX-1.1-Export bereitstellt. Die Einzelnen Anforderungen sind so formuliert, dass sie von einem Entwicklerteam umgesetzt und von einem Testteam überprüft werden können. Das Dokument dient als verbindliche Grundlage für die Entwicklung, die Qualitätssicherung und die Abnahme der Bibliothek.

Konkrete Ziele der Bibliothek:

- **Anforderungsanalyse und Spezifikation:** Alle Anforderungen werden nach den SOPHIST-Regeln eindeutig und prüfbar formuliert. Dies umfasst funktionale und nicht-funktionale Anforderungen, Anforderungen an die Datenschnittstellen und Randbedingungen, ergänzt um Qualitäts- und Risikobetrachtungen sowie ein objektorientiertes Analyse- und Entwurfsmodell.
- **Implementierung:** Es soll eine Java-Komponente ohne Benutzeroberfläche mit klaren Schnittstellen für Positions- und Kursdaten, konfigurierbarer Aufzeichnung, GPX-1.1-Export und optionaler What3Words-Anbindung implementiert werden.
- **Qualitätssicherung:** Es sollen nachvollziehbare Testfälle je fachlichem Modul, automatisiert über Maven ausführbar und mit reproduzierbaren Ergebnissen als Grundlage für die Abnahme geschrieben werden.
- **Lieferfähigkeit:** Der Quellcode soll direkt ausführbar sein. Ist dies nicht der Fall müssen alle Schritte zum starten des Programmes dokumentiert werden. Zudem sollen automatisierte Tests startbar mit `mvn test` sein.

Erfolgskriterien dieses Projekts:

- Die Komponente ist ohne UI lauffähig, die 15 verbindlichen Testfälle /TC010/ ... /TC150/ sind per `mvn test` ausführbar.
- Jede Anforderung /LF010/ ... /LF250/ und /LL010/ ... /LL080/ ist in der Traceability-Matrix (Anhang) einem /TC/, einer dokumentierten indirekten Abdeckung oder einem Konsolen-Demo-Nachweis zugeordnet.
- GPX-Ausgaben enthalten den GPX-1.1-Namespace und valide Zeitstempel im UTC-ISO-8601-Format.
- Keine Klassen aus UI-Frameworks werden referenziert.

1.2 Einsatzbereich

1.2.1 Problemstellung und fachlicher Hintergrund

Als fachliche Referenz dient die iOS-App *Kompass Professional*. Sie zeigt, wie eine Peilungsfunktion in der Praxis aussehen kann: Die App verrät, in welcher Richtung und Entfernung ein Zielpunkt von der eigenen Position aus liegt, sie führt aber nicht Schritt für Schritt dorthin, wie man es von einem Navigationssystem kennt. Nebenbei zeichnet sie den zurückgelegten Weg als GPS-Track auf, der sich als GPX-Datei exportieren lässt. Wenn in dieser Arbeit von Peilung die Rede ist, ist damit also die Berechnung von Richtung und Entfernung zwischen der aktuellen Position und einem Ziel gemeint.

Die App hat allerdings einen Haken: Ihre Peilungslogik ist so eng mit der Oberfläche und der Sensorhardware verwoben, dass man sie nicht losgelöst davon verwenden kann. Genau an dieser Stelle setzt die vorliegende Arbeit an. Die zu entwickelnde Bibliothek soll Zielrichtung (Azimut), Entfernung und Ordinalrichtung aus WGS84-Koordinaten, konsistent, reproduzierbar und unter klar definierten Eingangs- und Schnittstellenbedingungen, berechnen. Die Verarbeitung ist als Session organisiert, mit Start, Laufzeit und Abbruch. Wird eine Session abgebrochen, geht nichts verloren: Der bis dahin aufgezeichnete Track lässt sich weiterhin vollständig exportieren.

Zielbild der Bibliothek: Der Host liefert Positions- und Kursdaten, die Bibliothek berechnet daraus die Peilungsgrößen und zeichnet parallel einen GPS-Track auf, der als GPX 1.1 exportiert werden kann. Auf Wunsch lassen sich Koordinaten zusätzlich in eine What3Words-Adresse übersetzen.

1.2.2 Funktionsumfang

Im Kern umfasst die Arbeit Peilung, Track-Aufzeichnung und GPX-Export als eingebettete Java-Bibliothek ohne Benutzeroberfläche. Navigation, Kartendarstellung, Sensor-Hardware und Cloud-Dienste gehören nicht dazu. Die verbindliche technische Abgrenzung mit Scope- und Out-of-Scope-Liste steht in Kapitel 2 (Einschränkungen).

1.3 Glossar

Das folgende Glossar definiert die zentralen Fachbegriffe welche in diesem Dokument verwendet werden. Jede Begriffskarte folgt dabei demselben Schema mit den Feldern Definition, Abgrenzung, Gültigkeit, Bezeichnung/Symbol und Quellverweis.

1.3.1 Fachbegriffe Navigation und Geografie

Begriff	Peilung (Bearing)
Definition	Bestimmung der horizontalen Zielrichtung von einem Bezugspunkt zu einem Zielpunkt.
Abgrenzung	Keine Turn-by-Turn-Navigation.
Gültigkeit	Gültig für WGS84-Koordinaten auf der Erdoberfläche.
Bezeichnung / Symbol	
Quellverweis	Klärungsgespräch

Begriff	Geografischer Azimut
Definition	Horizontaler Winkel zwischen der geografischen Nordrichtung und der Großkreisrichtung zum Ziel.
Abgrenzung	
Gültigkeit	Gültig für Entfernungen > 0 m.
Bezeichnung / Symbol	Symbol: sym.alpha, Einheit: $^{\circ}$ (Grad).
Quellverweis	B. Hofmann-Wellenhof u. a., Navigation, Springer, https://link.springer.com/book/10.1007/978-3-7091-6078-7

Begriff	Kurs (Heading)
Definition	Orientierung der lokalen Horizontalebene relativ zu geografisch Nord, typischerweise als Winkel 0° - 360° .
Abgrenzung	
Gültigkeit	Gültig, wenn der Host verlässliche Kursinformation liefert.
Bezeichnung / Symbol	Symbol: sym.psi, Einheit: $^{\circ}$.
Quellverweis	B. Hofmann-Wellenhof u. a., Navigation, Springer, https://link.springer.com/book/10.1007/978-3-7091-6078-7

Begriff	Kursabweichung (relative Bearing)
Definition	Winkel zwischen aktuellem Kurs und der Zielrichtung, zentriert typischerweise in $[-180^\circ, +180^\circ]$.
Abgrenzung	
Gültigkeit	
Bezeichnung / Symbol	Symbol: sym.Delta, Einheit: $^\circ$.
Quellverweis	Berechnung in sym.Delta gemäß /LF050/ (optionaler Kurs aus /LF040/).

Begriff	Ordinalrichtung (Himmelsrichtung)
Definition	Vereinfachte 8er-Kompassangabe statt eines exakten Gradwerts.
Abgrenzung	
Gültigkeit	Wird aus dem Azimut abgeleitet und für eine leicht verständliche Richtungsanzeige genutzt.
Bezeichnung / Symbol	Werte: N, NE, E, SE, S, SW, W, NW.
Quellverweis	G. Hake u. a., Kartographie, https://books.google.com/books/about/Kartographie.html?id=MsggAAAAQBAJ und /LF050/

Begriff	Geografische Koordinate
Definition	Angabe eines Punktes auf der Erdoberfläche durch Breiten- und Längengrade.
Abgrenzung	
Gültigkeit	Gültig im WGS84-System.
Bezeichnung / Symbol	Notation: (lat, lon).
Quellverweis	ISO 19111

Begriff	Großkreisentfernung
Definition	Kürzeste Distanz zwischen zwei Ellipsoidpunkten entlang einer Großkreislinie auf der Kugelapproximation der Erde.
Abgrenzung	
Gültigkeit	Für kurze Distanzen < 20 km ist die sphärische Näherung üblicherweise ausreichend.
Bezeichnung / Symbol	Symbol: d, Einheit: m.
Quellverweis	B. Hofmann-Wellenhof u. a., Navigation, Springer, https://link.springer.com/book/10.1007/978-3-7091-6078-7

Begriff	Haversine-Formel
Definition	Sphärische Formel zur Berechnung der Zentriwinkelspanne und daraus der Großkreisdistanz aus zwei Breiten-/Längengraden.
Abgrenzung	
Gültigkeit	Gültig für Kugelradius R.
Bezeichnung / Symbol	Implementationsreferenz: lat1, lon1, lat2, lon2.
Quellverweis	B. Hofmann-Wellenhof u. a., Navigation, Springer, https://link.springer.com/book/10.1007/978-3-7091-6078-7

Begriff	ECEF (Earth-Centered, Earth-Fixed)
Definition	Rechtshändiges kartesisches Koordinatensystem mit Ursprung im Erdmittelpunkt (vereinfacht), Z-Achse durch den Nordpol.
Abgrenzung	
Gültigkeit	Interne Hilfskonstruktion für Distanzen im Raum optional, öffentliche API bleibt WGS84 lat/lon.
Bezeichnung / Symbol	Koordinaten X, Y, Z in Metern.
Quellverweis	ISO 19111 und B. Hofmann-Wellenhof u. a., Global Positioning System, Springer, https://link.springer.com/book/10.1007/978-3-7091-6199-9

Begriff	EPSG:4326
Definition	Geographisches CRS: Achsenfolge Breite (sym.phi), Länge (sym.lambda) in Grad auf WGS84-Ellipsoid.
Abgrenzung	
Gültigkeit	Gültig für alle Eingaben dieser Bibliothek, andere EPSG-Codes nur nach expliziter Erweiterung.
Bezeichnung / Symbol	CRS-Code als Metadaten-Hinweis exportierbar.
Quellverweis	EPSG Geodetic Parameter Dataset.

Begriff	Turn-by-Turn-Navigation
Definition	Schrittweise Routenführung entlang von Kanten eines Straßengraphen mit Manöveranweisungen.
Abgrenzung	
Gültigkeit	Explizit nicht Gegenstand der Peilungskomponente.
Bezeichnung / Symbol	
Quellverweis	Allgemeiner Sprachgebrauch, Abgrenzung Projektauftrag.

1.3.2 Begriffe zu Datenformaten und Protokollen

Begriff	GPX (GPS Exchange Format)
Definition	XML-basiertes Austauschformat für Wegpunkte, Routen und Tracks.
Abgrenzung	
Gültigkeit	Version 1.1 ist verbindlich für dieses Projekt.
Bezeichnung / Symbol	
Quellverweis	Topografix GPX 1.1 Schema

Begriff	Trackpunkt
Definition	XML-Element innerhalb eines GPX-Tracks, das mindestens Breite, Länge und Zeit beinhaltet.
Abgrenzung	
Gültigkeit	Gültig innerhalb eines Tracks.
Bezeichnung / Symbol	Attribute: `lat`, `lon`.
Quellverweis	GPX 1.1 Spezifikation

Begriff	HDOP (Horizontal Dilution of Precision)
Definition	Dimensionsloses Maß für die geometrische Güte der Satellitenkonstellation, niedriger ist typischerweise besser.
Abgrenzung	
Gültigkeit	Optional im Datenmodell und in GPX (keine automatische Verwerfung beim Einlesen) (/LF030/, /LD040/).
Bezeichnung / Symbol	
Quellverweis	B. Hofmann-Wellenhof u. a., Global Positioning System, Springer, https://link.springer.com/book/10.1007/978-3-7091-6199-9

Begriff	UTC und `Instant`
Definition	UTC ist die einheitliche Zeitbasis ohne Sommerzeit, `java.time.Instant` beschreibt einen exakten Zeitpunkt auf dieser Skala.
Abgrenzung	
Gültigkeit	Interne Zeitstempel laufen durchgängig über `Instant` (Ausgabe nach GPX erfolgt als ISO-8601 in UTC).
Bezeichnung / Symbol	Im Code: `ClockPort.instant()` und `DateTimeFormatter.ISO_INSTANT`.
Quellverweis	ISO 8601, Java SE Date-Time-Package.

Begriff	UTF-8
Definition	Byte-Kodierung für Unicode-Zeichen (verbindlich für GPX-XML in diesem Projekt).
Abgrenzung	
Gültigkeit	Alle serialisierten GPX-Dokumente.
Bezeichnung / Symbol	Kennung in XML-Deklaration optional, Bytes konsistent.
Quellverweis	Unicode Standard Annex #15.

1.3.3 Begriffe zu Session und Systemverhalten

Begriff	Session
Definition	Eine Session ist eine zeitlich begrenzte Instanz zur Speicherung von Zustandsinformationen eines Benutzers oder Prozesses.
Abgrenzung	Abzugrenzen ist die Session von einer einzelnen Anfrage sowie von persistenten Objekten, die dauerhaft gespeichert werden. Eine Session ist temporär und zustandsbehaftet.
Gültigkeit	Die Gültigkeit einer Session erstreckt sich über die Dauer der Interaktion und endet durch Timeout.
Bezeichnung / Symbol	Session-ID
Quellverweis	H. Balzert, Lehrbuch der Softwaretechnik, Springer, https://link.springer.com/book/10.1007/978-3-8274-2247-7

Begriff	Abbruchverhalten (Abort)
Definition	Vorzeitiges Beenden einer aktiven Session durch den Host, ohne die bis dahin erfassten Daten zu verlieren.
Abgrenzung	
Gültigkeit	Session wechselt auf `Lifecycle.ABORTED`. Rückgabe erfolgt als `GpxResult` (UTF-8-Bytes plus String-Zugriff) gemäß /LF070/ und /LF160/.
Bezeichnung / Symbol	Persistenz beim Abort erfolgt nur bei aktivem `persistOnAbort`.
Quellverweis	Klärungsgespräch, /LF070/

Begriff	Observer / Listener
Definition	Benachrichtigen registrierte Beobachter über Zustandsänderungen.
Abgrenzung	
Gültigkeit	Synchronität gemäß Konfiguration /LF080/.
Bezeichnung / Symbol	Java-Pattern: `java.util.EventListener`-Familie.
Quellverweis	/LF080/

Begriff	Trackoptimierung
Definition	Austauschbare Algorithmen hinter gemeinsamer Schnittstelle zur Reduktion oder Vereinfachung von Trackpunkten.
Abgrenzung	
Gültigkeit	Nur optional vor GPX-Export über `SessionConfig.builder().addOptimizer(...)` mit Schnittstelle `TrackOptimizer` (leere Liste = unveränderter Roh-track), /LF170/ bis /LF200/.
Bezeichnung / Symbol	
Quellverweis	/LF170/ bis /LF200/

Begriff	Soft-Limit / Hard-Limit (Punktbudget)
Definition	Zweistufige Begrenzung der gespeicherten Trackpunkte: erst Warnung, dann harter Stopp.
Abgrenzung	
Gültigkeit	Konfigurierbar über `softLimitPoints` und `hardLimitPoints`, `0` bedeutet deaktiviert (/LF110/, /LF130/).
Bezeichnung / Symbol	Code: `TrackAcceptResult.Flag.SOFT_LIMIT_WARN`, `TrackAcceptResult.Flag.HARD_LIMIT_REACHED`.
Quellverweis	Ressourcen-Schutz in eingebetteten Systemen (projektspezifisch).

Begriff	Overflow-Modus (Punktbudget)
Definition	Verhalten bei Erreichen des Hard-Limits: Aufzeichnung stoppen.
Abgrenzung	
Gültigkeit	Konfigurationsabhängig /LF130/; aktuell nur `OverflowMode.STOP`.
Bezeichnung / Symbol	Enum: `OverflowMode.STOP` (Hard-Limit erreicht, Aufzeichnung stoppt).
Quellverweis	Interne Spezifikation.

1.3.4 Begriffe zu Entwurfsmustern und Architektur

Begriff	API (Application Programming Interface)
Definition	Programmierschnittstelle, über die externe Systeme mit der Peilungskomponente interagieren.
Abgrenzung	
Gültigkeit	Öffentliche Methoden der Bibliothek.
Bezeichnung / Symbol	
Quellverweis	I. Sommerville, Software Engineering, https://books.google.com/books/about/Software_Engineering_Global_Edition.html?id=W_LjCwAAQBAJ

Begriff	Builder (Konfigurationsaufbau)
Definition	Muster, mit dem man eine Konfiguration Schritt für Schritt baut statt mit einem langen Konstruktor.
Abgrenzung	
Gültigkeit	Im Projekt über `SessionConfig.builder()`, beim Start werden die relevanten Parameter als `RecordingParameters` eingefroren (/LD020/).
Bezeichnung / Symbol	Beispiel: `SessionConfig.builder().segmentGapThreshold(java.time.Duration.ofMinutes(5)).build()`
Quellverweis	E. Gamma u. a., Design Patterns, https://books.google.com/books/about/Design_Patterns.html?id=6oHuKQe3TjQC

Begriff	Immutability
Definition	Ein Objekt bleibt nach der Erzeugung unverändert, für Änderungen wird ein neues Objekt erstellt.
Abgrenzung	
Gültigkeit	Trifft hier u. a. auf `SessionConfig` und `GpsPoint` zu.
Bezeichnung / Symbol	Hilft bei Nebenläufigkeit, weil Leser keine veränderbaren Zwischenzustände sehen.
Quellverweis	J. Bloch, Effective Java, https://books.google.com/books/about/Effective_Java.html?id=piOyzYqeZGgC

Begriff	Thread-Sicherheit
Definition	Eigenschaft eines Systems, bei paralleler Ausführung durch mehrere Threads korrekt zu funktionieren.
Abgrenzung	
Gültigkeit	Relevant für Observer, Immutability und Eventverarbeitung.
Bezeichnung / Symbol	
Quellverweis	B. Goetz u. a., Java Concurrency in Practice, https://books.google.com/books/about/Java_Concurrency_in_Practice.html?id=EK43StEVfJIC

Begriff	Semantische Exception
Definition	Fehlerklasse mit maschinenlesbarem `errorCode` und kontextualisierter Message.
Abgrenzung	
Gültigkeit	Öffentliche API der Bibliothek.
Bezeichnung / Symbol	Klasse `ValidationException`, Feld `ErrorCode errorCode`, Zugriff über `errorCode()`.
Quellverweis	/LF090/ und J. Bloch, Effective Java, https://books.google.com/books/about/Effective_Java.html?id=piOyzYqeZGgC

1.3.5 Begriffe zu Sicherheit

Begriff	Path Traversal
Definition	Sicherheitsproblem, bei dem ein Pfad absichtlich aus dem erlaubten Zielordner ausbricht (z.\ B. mit `..`).
Abgrenzung	
Gültigkeit	Beim Export verpflichtend zu verhindern (/LF250/).
Bezeichnung / Symbol	Umsetzung in `SafeFileSink`: Pfad normalisieren und gegen erlaubtes Basisverzeichnis prüfen.
Quellverweis	OWASP Path Traversal

Begriff	XXE (XML External Entity)
Definition	Angriffsklasse beim XML-Parsing über externe Entities.
Abgrenzung	
Gültigkeit	Die Komponente erzeugt XML selbst und escaped Inhalte (/LL050/), das XXE-Risiko liegt vor allem bei fremden Parsern, die diese XML später lesen.
Bezeichnung / Symbol	Empfehlung: DTD/External-Entity-Verarbeitung im konsumierenden Parser deaktivieren.
Quellverweis	W3C XML Recommendation

Begriff	Atomares Schreiben
Definition	Datei wird zuerst temporär geschrieben und dann in den Zielpfad verschoben, um halbfertige Dateien zu vermeiden.
Abgrenzung	
Gültigkeit	Wichtig für die Konsistenz bei Exporten.
Bezeichnung / Symbol	Temporäre Datei (`*.tmp`).
Quellverweis	Anforderung /LF150/.

1.3.6 Begriffe zu Algorithmen

Begriff	Douglas-Peucker-Algorithmus
Definition	Algorithmus zur Linienvereinfachung: Punkte mit geringer Abweichung von der Verbindungslinie werden rekursiv entfernt.
Abgrenzung	
Gültigkeit	Im Projekt mit metrischer Toleranz in Metern und lokaler Tangentialebene als Näherung (/LF200/).
Bezeichnung / Symbol	Parameter im Code: `epsilonM`.
Quellverweis	D. Douglas, T. Peucker, Algorithms for the reduction of the number of points required to represent a digitized line or its caricature, 1973.

Begriff	Kollinearitätsreduktion
Definition	Reduktion fast kollinearer Punktfolgen auf Start- und Endpunkt innerhalb eines Winkel-/Seitenbandes.
Abgrenzung	
Gültigkeit	Explizit keine Garantie für enge Kurvenradien /LF190/.
Bezeichnung / Symbol	Heuristikparameter: `toleranceM` in `LineCollinearityOptimizer`.
Quellverweis	Klärungsgespräch und /LF190/.

1.3.7 Begriffe zu Methodik und Qualitätssicherung

Begriff	Anforderung
Definition	Beschriebene Eigenschaft oder Fähigkeit, die ein System erfüllen muss oder soll.
Abgrenzung	
Gültigkeit	Gültig für alle /LF.../, /LL.../, /LD.../ Artefakte.
Bezeichnung / Symbol	Typen: funktional, nicht-funktional.
Quellverweis	ISO/IEC/IEEE 29148.

Begriff	Traceability (Rückverfolgbarkeit)
Definition	Nachvollziehbare Verbindung zwischen Anforderung, Design, Code und Test.
Abgrenzung	
Gültigkeit	Wichtig über den gesamten Lebenszyklus, besonders bei `/LF.../` und `/TC.../`.
Bezeichnung / Symbol	Im Projekt als Matrix in `implementation/TRACEABILITY.md`.
Quellverweis	K. Pohl, Requirements Engineering Fundamentals, https://books.google.com/books/about/Requirements_Engineering_Fundamentals.html?id=GbsiTwEACAAJ

Begriff	Property "deterministisch"
Definition	Gleiche Eingaben und gleiche konfigurierte Abhängigkeiten (Clock) erzeugen identische Ausgaben.
Abgrenzung	
Gültigkeit	Gilt für Kern-Pfade ohne bewusst randomisierte Algorithmen.
Bezeichnung / Symbol	
Quellverweis	/LL070/.

1.3.8 Begriffe zu Werkzeugen und Frameworks

Begriff	JVM (Java Virtual Machine)
Definition	Laufzeitumgebung, die Java-Bytecode plattformunabhängig ausführt. Die Bibliothek setzt eine JVM voraus, stellt selbst jedoch keine bereit.
Abgrenzung	
Gültigkeit	Ziel-Kompatibilität: Java 11 (LTS), die Bibliothek wird als reines Source-Artefakt ohne ausführbares JAR ausgeliefert.
Bezeichnung / Symbol	Die JVM-Version des Host-Projekts muss ≥ 11 sein (niedrigere Versionen werden nicht unterstützt).
Quellverweis	Systemvoraussetzungen, /LL030/

Begriff	Maven
Definition	Standardwerkzeug für Build, Testlauf und Abhängigkeitsverwaltung in Java-Projekten.
Abgrenzung	
Gültigkeit	Im Projekt zentraler Einstieg für Build und Tests (<code>`mvn test`</code> , /LL040/).
Bezeichnung / Symbol	Artefakte werden über <code>`groupId:artifactId:version`</code> aufgelöst.
Quellverweis	Apache Maven Project Dokumentation, https://maven.apache.org/

Begriff	Maven-Profil (<code>`w3w`</code>)
Definition	Optionale Build-Konfiguration, die What3Words-Clientabhängigkeiten aktiviert.
Abgrenzung	
Gültigkeit	Standardbuild ohne Profil bleibt offline-fähig.
Bezeichnung / Symbol	Aktivierung: <code>`mvn -Pw3w test`</code> (konzeptionell).
Quellverweis	Apache Maven Documentation, https://maven.apache.org/guides/introduction/introduction-to-profiles.html

Begriff	SLF4J (Simple Logging Facade for Java)
Definition	Einheitliche Logging-Schnittstelle, die konkrete Logger-Backends austauschbar macht.
Abgrenzung	
Gültigkeit	Die Komponente bindet Logging über den Adapter <code>`Slf4jLoggerAdapter`</code> ein (/LL060/).
Bezeichnung / Symbol	Typischer Einstieg: <code>`LoggerFactory.getLogger(...)`</code> .
Quellverweis	https://www.slf4j.org/

Begriff	JUnit 5
Definition	Modernes Java-Testframework für Unit- und Integrationsnahe Tests.
Abgrenzung	
Gültigkeit	In diesem Projekt Grundlage der Modul-Tests, aktuell vor allem mit `@Test`.
Bezeichnung / Symbol	Bietet u. a. Assertions, Lifecycle-Hooks und Erweiterungen.
Quellverweis	https://junit.org/junit5/

Begriff	JaCoCo
Definition	Code-Coverage-Tool für Java, Integration über Maven-Plugin.
Abgrenzung	
Gültigkeit	Optionales Build-Artefakt (kein Abgabe-Gate).
Bezeichnung / Symbol	Report: `target/site/jacoco/index.html`.
Quellverweis	https://www.jacoco.org/jacoco/trunk/doc/

Begriff	Checkstyle
Definition	Werkzeug für statische Stil- und Strukturprüfungen im Java-Code.
Abgrenzung	
Gültigkeit	Im Build integriert, die aktuelle Projektkonfiguration ist bewusst schlank gehalten.
Bezeichnung / Symbol	Konfiguration in `implementation/checkstyle.xml`.
Quellverweis	https://checkstyle.org/

Begriff	ArchUnit
Definition	Bibliothek zur Überprüfung architektonischer Regeln im JUnit-Test (z.\ B. keine AWT-Imports).
Abgrenzung	
Gültigkeit	Optional zur Absicherung von `/LL030/`.
Bezeichnung / Symbol	Regel: `noClasses().that()...`
Quellverweis	https://www.archunit.org/

Begriff	Jimfs
Definition	In-Memory-Dateisystem-Implementierung für JDK `java.nio.file`, geeignet für Tests.
Abgrenzung	
Gültigkeit	Nur Testscope.
Bezeichnung / Symbol	Google Jimfs Projekt.
Quellverweis	https://github.com/google/jimfs

Begriff	Property-Based Testing (jqwik)
Definition	Testansatz, bei dem allgemeine Eigenschaften statt einzelner Beispiele geprüft werden.
Abgrenzung	
Gültigkeit	Sinnvoll für Invarianten wie Distanzsymmetrie oder Grenzfälle bei Winkeln.
Bezeichnung / Symbol	Kann optional mit jqwik umgesetzt werden.
Quellverweis	https://jqwik.net/

Begriff	Referenzimplementierung (Test)
Definition	Unabhängige, minimal gehaltene Berechnung derselben Größe zum Abgleich von Toleranzen.
Abgrenzung	
Gültigkeit	Testcode, nicht Produktivpfad.
Bezeichnung / Symbol	`ReferenceHaversine` (Bezeichner frei wählbar).
Quellverweis	/LL020/ Testpraxis.

Begriff	What3Words (W3W)
Definition	Dienst, der Koordinaten in ein Drei-Wort-Format umwandelt.
Abgrenzung	
Gültigkeit	Optionaler Teil des Systems (/LF210/), Ergebnisse werden gecacht (/LF220/).
Bezeichnung / Symbol	
Quellverweis	what3words Limited, Klärungsgespräch

Begriff	System (Softwarekomponente)
Definition	Abgegrenzter Teil eines Softwaresystems mit klar definierten Schnittstellen und Verantwortlichkeiten.
Abgrenzung	
Gültigkeit	Hier: die Peilungskomponente als eigenständige Bibliothek.
Bezeichnung / Symbol	
Quellverweis	L. Bass u. a., Software Architecture in Practice, https://books.google.com/books?id=-II73rBDXCYC

Begriff	Ist-/Soll-Vergleich (Referenz-App)
Definition	Qualitative Gegenüberstellung der Referenz-App mit der Java-Bibliothek ohne pixelgenaue UI-Nachbildung.
Abgrenzung	
Gültigkeit	Dient der Motivation und Begriffsableitung, nicht als normative Spezifikation der UI.
Bezeichnung / Symbol	
Quellverweis	Kompass Pro App, Klärungsgespräch

Begriff	Genauigkeit
Definition	Accuracy beschreibt die Nähe zum wahren Wert, Precision die Wiederholgenauigkeit von Messungen.
Abgrenzung	
Gültigkeit	Relevant für GPS-Datenbewertung.
Bezeichnung / Symbol	
Quellverweis	R. Lerch, Elektrische Messtechnik, Springer, https://link.springer.com/book/10.1007/978-3-540-75727-6

1.4 Überblick

Der Aufbau des Dokuments orientiert sich an IEEE 830 und gliedert sich in drei Hauptkapitel:

Kapitel 1 – Einleitung definiert Zweck und Einsatzbereich der Bibliothek, legt mit dem Glossar eine einheitliche Terminologie fest und gibt einen Überblick über den Aufbau des Dokuments.

Kapitel 2 – Allgemeine Beschreibung ordnet die Bibliothek in ihr Umfeld ein: Es beschreibt die System- und die physische Sicht sowie die externen Kommunikationspartner, benennt die Hauptfunktionen und dokumentiert systemweite Einschränkungen, Annahmen und Abhängigkeiten.

Kapitel 3 – Spezifische Anforderungen bildet den normativen Kern des Dokuments. Es umfasst die funktionalen Spezifikationen (/LF.../) mit zugehörigen Aktivitätsdiagrammen (Kap. 3.1), die nicht-funktionalen Anforderungen (/LL.../), die externen Schnittstellen, die Performance-Anforderungen sowie das Qualitätsmodell nach ISO/IEC 25010.

Der **Anhang** vervollständigt die Spezifikation um die Traceability-Matrix, das objektorientierte Analyse- und Entwurfsmodell, den Referenzalgorithmus der Peilung sowie Literatur- und Normenverweise.

2 Allgemeine Beschreibung

Dieses Kapitel betrachtet die Java-Peilungskomponente aus der Übersicht und beschäftigt sich damit, wie sie sich in das technische Umfeld einfügt, welche Hauptfunktionen sie bietet, welche Nutzercharakteristiken gelten und welche systemweiten Einschränkungen, Annahmen und Abhängigkeiten festgelegt sind.

2.1 Einbettung

2.1.1 Systemsicht und Systemgrenze

Die Java-Peilungskomponente ist eine reine Bibliothek ohne eigene Benutzeroberfläche. Sie wird in den Prozess einer Host-Anwendung eingebettet und läuft in deren JVM. Es existiert kein eigener Prozess, kein eigenständiger Server und kein Nachrichtensystem.

Die Systemgrenze umschließt ausschließlich die Java-Peilungskomponente. Sensordaten (GNSS, Kompass) werden nicht direkt gelesen. Die Host-Anwendung ist verantwortlich für Sensorfusion und Gerätezugriff und übergibt fertige Messwerte über die öffentliche API.

Externe Kommunikation findet ausschließlich über zwei optionale Kanäle statt: HTTPS zur W3W-API und Dateisystemzugriffe für den GPX-Export. Beide Kanäle sind abschaltbar und die Kernfunktion (Peilung, Track-Aufzeichnung) läuft ohne Netzwerkverbindung.

Externe Schnittstelle	Richtung	Zweck
Host-Anwendung	eingehend	Steuerung des Session-Lebenszyklus sowie die Übergabe von Positions- und Kursdaten
Listener / Result	ausgehend	Snapshots, Statuswechsel, Fehlerereignisse und GPX-Rückgabe an den Host
Dateisystem	ausgehend, optional	GPX-Persistenz nur bei expliziter Konfiguration durch den Host
W3W-HTTP	ausgehend, optional	Reverse-Lookup mit lokalem Cache, ist ohne Netzwerk vollständig deaktivierbar

Tabelle 1: Externe Schnittstellen der Bibliothek und ihre Kommunikationsrichtungen.

2.1.2 Schichtenarchitektur

Das System ist in vier Subsysteme gegliedert (API, Domain, Ports, Infrastruktur). Die API orchestriert Fachlogik und injizierte Port-Interfaces; konkrete Adapter implementieren die Ports. Zyklische Abhängigkeiten zwischen Domain und Infrastruktur sind ausgeschlossen.

Subsystem	Kernverantwortung	Bereitgestellte Dienste
API / Application Service	Use-Case-Orchestrierung, Session-Verwaltung und Fehlerbehandlung	Session starten, beenden und abbrechen und konsistenten Zustandschnappschuss liefern
Domain Core	Fachlogik ohne I/O-Abhängigkeiten	Azimut, Distanz, Ordinalrichtung, Rohspeicher, Validierung, Segmentierung und Export-Optimierer
Ports	Abstraktion technischer Abhängigkeiten	Stabile Schnittstellen für GPX, Zeitgeber, Dateisystem, Logging und W3W
Infrastruktur / Adapter	Konkrete Implementierung der Ports	XML-Serialisierung, Datei-I/O, HTTP-Client, Systemuhr, Logging-Backend

Tabelle 2: Subsysteme und ihre Verantwortlichkeiten.

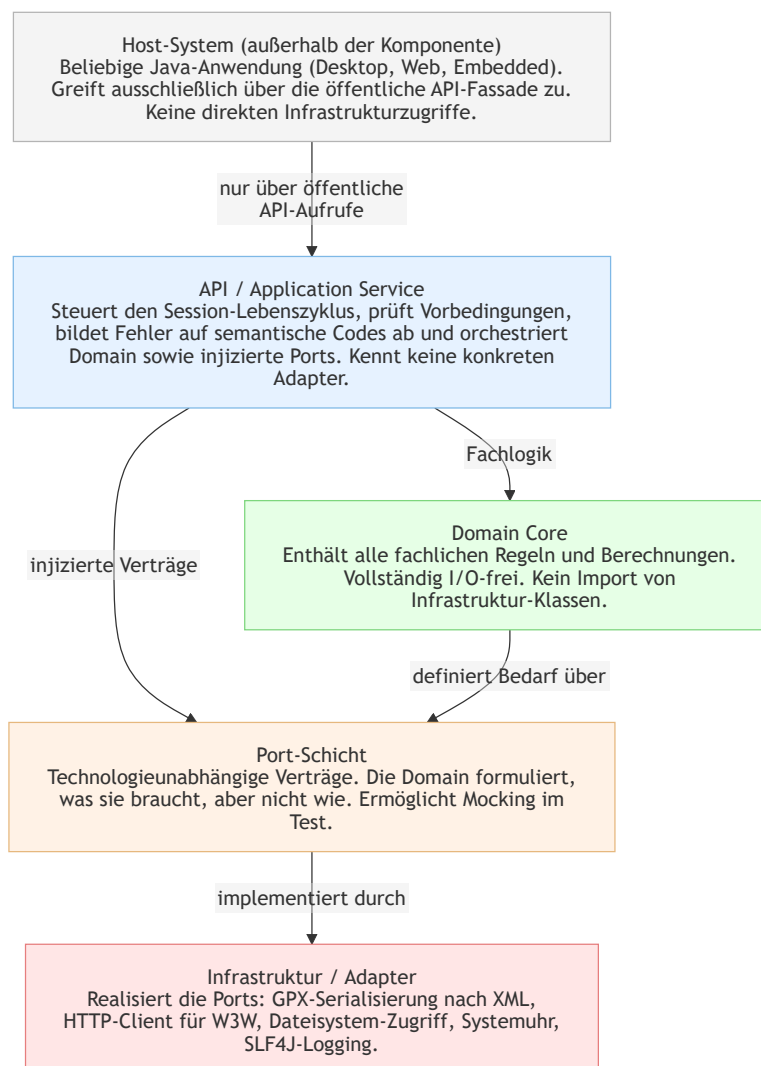


Abbildung 1: Architekturschichten und ihr Kontrollfluss

2.1.3 Kommunikationsregeln

Von	Nach	Erlaubt?	Begründung
Host	API	Ja	Die API-Fassade ist der einzige definierte Einstiegspunkt.
Host	Domain	Nein	Direkter Zugriff würde die Kapselungswirkung der API-Fassade aushebeln.
API	Domain	Ja	Orchestrierung der Fachlogik ist Kernaufgabe der API-Schicht.
API	Ports	Ja	Injizierte Port-Interfaces (<code>GpxWriterPort</code> , <code>FileSinkPort</code> , <code>ClockPort</code> , ...).
API	Infrastruktur	Nein	Keine technischen Details im Application Service und nur Port-Interfaces.
Domain	Infrastruktur	Nein	Fachlogik bleibt I/O-frei und Infrastruktur wird über Ports entkoppelt.
Infrastruktur	Domain	Nein	Zirkuläre Abhängigkeit würde die Testbarkeit zerstören.

Tabelle 3: Erlaubte und verbotene Abhängigkeiten zwischen den Subsystemen.

2.1.4 Physische Sicht

Die Komponente wird als Bibliothek in den Host-Prozess eingebettet und läuft in dessen JVM. Für Integrationstests sind lokale Mocks der Ports ausreichend. Die Offline-Fähigkeit sichert den Betrieb auch ohne Netzwerkzugang.

2.2 Funktionen

Die folgende Tabelle fasst die Hauptfunktionsgruppen der Bibliothek zusammen. Die genaue Spezifikation der einzelnen Anforderungen folgt in Kapitel 3.

Funktionsgruppe	Beschreibung	Anforderungen
Session-Lebenszyklus	Start, laufende Aufzeichnung und Abbruch einer Peilungs-Session mit UUID-Identifikation.	/LF010/-/ LF090/
Peilung und Kurs	Berechnung von geografischem Azimut, Entfernung (Haversine) und diskreter Himmelsrichtung und optionale Kursabweichung.	/LF030/-/ LF050/
GPS-Track-Aufzeichnung	Kontinuierlicher Rohspeicher validierter GPS-Fixes, Segmentierung bei Zeitlücken und zweistufiges Punktbudget.	/LF100/-/ LF130/
GPX-Export	GPX-1.1-konformer Export als byte[]; optionales atomares Dateischreiben.	/LF140/-/ LF160/
Track-Optimierung	Austauschbare Strategie-Algorithmen: n-ter Punkt, Mindestabstand, Geraden-Heuristik, Douglas-Peucker.	/LF170/-/ LF200/
What3Words-Integration	Optionaler Reverse-Lookup mit Cache via W3wClientPort.	/LF210/, / LF220/
Validierung und Sicherheit	Koordinaten- und Zeitstempelprüfung, Path-Traversal-Schutz und XML-Escaping.	/LF230/-/ LF250/
Betrieb und Qualität	Deterministische Tests, strukturiertes Logging, reproduzierbarer Build.	/LL010/-/ LL080/

Tabelle 4: Hauptfunktionsgruppen der Java-Peilungskomponente mit Verweis auf Anforderungen.

Weitere Entwurfsprinzipien:

Prinzip	Entwurfsentscheidung	Messbarer Nutzen
Hohe Kohäsion	Module sind strikt nach fachlicher Verantwortung getrennt.	Anpassungen am GPX-Format bleiben lokal auf den <code>bearing-adapter-gpx</code> begrenzt.
Schwache Kopplung	Komponenten kommunizieren ausschließlich über APIs statt über konkrete Implementierungen.	Unterkomponenten lassen sich flexibler austauschen, was Mocking für Unit-Tests vereinfacht.
Information Hiding	Domänen-Logik wird hinter einer Fassade gekapselt. Daten fließen als unveränderliche Value Objects.	Die interne Struktur ist geschützt; externe Aufrufer können den Systemzustand nicht schädigen.
Separation of Concerns	Saubere Trennung zwischen Kernlogik und Infrastruktur (Dateizugriff, Netzwerk-I/O).	Fachlogik lässt sich ohne Infrastruktur-Overhead testen und umgekehrt.
Wiederverwendbarkeit	Einsatz des Strategy-Patterns für Policies und Optimizer.	Neue Algorithmen lassen sich ohne Eingriffe in den bestehenden Kontrollfluss integrieren.

Tabelle 5: Entwurfsprinzipien und deren messbarer Nutzen.

2.3 Benutzerprofile

Als Bibliothek ohne eigene Benutzeroberfläche hat die Peilungskomponente keine direkten Endnutzer. Nach IEEE 830 umfasst dieser Abschnitt die Nutzercharakteristiken: für jede Nutzergruppe werden Erfahrung und Fachkenntnis, technische Voraussetzungen sowie Erwartungen an die Bibliothek beschrieben. Systemgrenze, externe Schnittstellen und Integrationspflichten des Hosts sind in Kap. 2.1 (Einbettung), Kap. 2.4 (Einschränkungen) und Kap. 3.3 (Externe Schnittstellen) spezifiziert; Use-Case-Akteure stehen je Anforderung in den /LF.../-Karten.

2.3.1 Nutzercharakteristiken

Nutzergruppe	Bezug	Erfahrung und Fachkenntnis	Technische Voraussetzungen	Erwartung an die Bibliothek
Host-Entwickler/in	direkt	Softwareentwicklung; Erfahrung mit Java-Bibliotheken und eingebetteten Komponenten.	Java 11 oder höher, Integration über die öffentliche API, optional Maven, SLF4J und W3W-Konfiguration durch den Host.	Stabile, dokumentierte API, <code>ValidationException</code> mit maschinenlesbarem <code>ErrorCode</code> bei Eingabefehlern und reproduzierbares Session-Verhalten.
Endnutzer/in	indirekt	Keine Kenntnis der Bibliothek, nutzt Peilungs- oder Outdoor-Funktionen über die Host-App.	Kein direkter Zugriff auf die Bibliothek.	Zuverlässige Peilungswerte und korrekter GPX-1.1-Export in der Host-App.
Betrieb (Host-Team)	indirekt	Betrieb eingebetteter Java-Anwendungen und Lesen strukturierter Logausgaben.	Bindet ein SLF4J-Backend im Host-Prozess.	Strukturierte Logausgabe und Listener-Fehler beeinträchtigen den Session-Zustand nicht (/LF080/).

Tabelle 6: Nutzergruppen der Bibliothek mit Erfahrung, Voraussetzungen und Erwartungen.

2.3.2 Projektkontext (Stakeholder)

Die folgende Matrix ergänzt die Nutzercharakteristiken um projektbezogene Rollen und deren Einfluss auf diese Arbeit. Sie ist nicht Teil der Laufzeit-Spezifikation.

Rolle	Erwartung / Interesse	Einfluss
Dozent / Prüfer	Nachweis der Vorlesungsinhalte, lauffähiger Code, formalsaubere Dokumentation nach IEEE-/SOPHIST-Standard.	hoch
Studierenden-team	Wartbare, erweiterbare Architektur, klare Testbarkeit und nachvollziehbare Anforderungen.	hoch
Host-Entwickler/in	Stabile, vollständig dokumentierte API, klare Fehlersemantik über <code>ValidationException</code> und <code>ErrorCode</code> .	hoch
Endnutzer/in (indirekt)	Zuverlässige Peilungswerte und korrekte GPX-Ausgabe in der Host-Anwendung.	mittel
Betrieb	Strukturiertes Logging auf WARN-Level, keine stillen Fehler und deterministisches Verhalten.	niedrig-mittel

Tabelle 7: Stakeholder der Java-Peilungskomponente mit Erwartungen und Einfluss.

2.4 Einschränkungen

Die folgende Auflistung definiert den technischen Funktionsumfang als verbindliche Systemgrenze. Sie präzisiert Integrationsschnittstellen, Session-Verhalten, Eingabevalidierung, Exportformate und Qualitätssicherung.

2.4.1 Im Scope

- Berechnung von Zielrichtung (Azimut), Entfernung und Himmelsrichtung (Ordinalrichtung) auf WGS84-Basis und bei verfügbarem Kurswert zusätzlich Kursabweichung.
- Öffentliche Java-API zur Übergabe von Positionsdaten und Metadaten (Zeitstempel, Geschwindigkeit, Genauigkeit) durch den Host.
- Session-Lebenszyklus (Start, Update, Complete/Abort) und nach Abort bleiben erfasste Track-Daten exportierbar.
- Konfigurierbare Aufzeichnung: Punktbudget (Soft-/Hard-Limit), Segmentierung bei Zeitlücken, Validierung von Koordinaten und Zeitstempeln; Update-Frequenz steuert der Host.
- GPX-1.1-Export als String oder Bytefolge.
- Vor dem Export wählbare Optimierungsstrategien (n-ter Punkt, Mindestabstand, Geraden-Heuristik, Douglas-Peucker) sowie optionale What3Words-Auflösung mit lokalem Cache.
- Automatisierte Unit-Tests für geografische Berechnungen, Optimierungsalgorithmen und GPX-Serialisierung.

2.4.2 Außerhalb des Scopes

- Echtzeit-Optimierung eingehender Track-Punkte während der laufenden Aufzeichnung.
- Magnetische Peilung und automatische Deklinationskorrektur.
- Kartendarstellung, Map-Matching, Routing und allgemeines Geocoding (What3Words ausgenommen).
- Direkte Hardwareanbindung (GNSS-Treiber, Sensorfusion) → Messwerte liefert ausschließlich der Host.
- Vorgegebene Persistenzpfade für GPX-Export → Speicherort und Dateiverwaltung obliegen dem Host.
- Cloud-Persistenz, Benutzer- und Rechteverwaltung.

2.5 Annahmen und Abhängigkeiten

2.5.1 Annahmen

ID	Annahme	Einfluss auf Anforderungen und Verhalten	Folge bei Verletzung
A1	Der Host liefert fertige Positions- und Kursdaten über die öffentliche API. Die Bibliothek liest keine Sensoren und führt keine Sensorfusion aus.	Systemgrenze und Funktionsumfang (Peilung und Track aus bereitgestellten Messwerten) bleiben definiert.	Ohne valide Host-Daten sind Peilung und Aufzeichnung nicht sinnvoll nutzbar, dies liegt außerhalb der Verantwortung der Bibliothek.
A2	Nur der Host steuert die Aufruffrequenz;	Roh-Trackfüllung, Punktbudget und Segmentierung	Andere Taktungsmodelle würden die dokumentierten

	die Bibliothek dünnt den eingehenden Datenstrom nicht von sich aus aus.	folgen der vom Host gewählten Updatefrequenz.	Erwartungen verletzen und Anpassungen am SRS nach sich ziehen.
A3	Geografische Berechnungen und Eingaben beziehen sich auf WGS84-konforme Koordinaten, wie in der Spezifikation vorausgesetzt.	Azimut, Distanz und Exportformate bleiben konsistent zur fachlichen Definition.	Abweichende Bezugssysteme erfordern eine Überarbeitung der Anforderungen und Algorithmen.
A4	Die Host-JVM unterstützt mindestens Java 11.	Bytecode, APIs und Tooling der Bibliothek sind darauf ausgelegt.	Ältere JVMs werden nicht unterstützt; ein Wechsel der Mindestversion wäre eine System-Änderung.
A5	Integratoren und Tests können eine Clock injizieren, um zeitabhängiges Verhalten deterministisch abzusichern.	Reproduzierbare Tests und nachvollziehbare Session-Zeitlogik.	Ohne geeignete Zeitquelle können zeitbasierte Regeln in Tests schwer stabil gehalten werden.

Tabelle 8: Übersicht der getroffenen Systemannahmen

2.5.2 Abhängigkeiten

Kategorie	Abhängigkeit	Details und Konsequenz
Host	Konfiguration (W3W)	API-Schlüssel und Adapter-Konfiguration bei Nutzung von What3Words. Ohne Schlüssel oder Netz gelten dokumentierte Fallbacks; volle W3W-Funktionalität entfällt.
Host	Dateisystem (GPX)	Zielpfade, ggf. <code>SafeFileSink</code> und Schreibrechte. Bei Verletzung schlägt GPX-Persistenz fehl oder wird abgelehnt; Fehler über definierte Mechanismen sichtbar.
Host	Logging	SLF4J-API der Bibliothek; konkretes Backend bindet der Host. Ohne Binding keine oder unerwartete Logausgabe, die Fachlogik unberührt.
Host	Netzwerk	HTTPS zur W3W-API nur bei aktivierter Nutzung; Erreichbarkeit und TLS-Vertrauen im Host-Umfeld. Eingeschränkter oder ausgefallener Reverse-Lookup und Cache-Update.
Laufzeit	Java SE	Version 11
Build	Apache Maven	Projekt unter <code>implementation</code> /und Abhängigkeiten über Maven Central. Kompilieren, Testen, Packen und Änderungen am Build können das SRS ergänzen lassen.

Laufzeit	slf4j-api	2.0.12 (Parent-POM). Fassade für Logging. Versionwechsel kann Integrationshinweise betreffen.
Test	JUnit 5, AssertJ, Mockito, Jims	Versionen 5.10.2, 3.25.3, 5.11.0, 1.3.0. Nur für automatisierte Tests (nicht Teil der Host-Laufzeit).
Extern	What3Words-API	Optional über HTTPS. Profil w3w steuert optionale Integrationstests im Build. Verfügbarkeit und API-Änderungen liegen außerhalb der Bibliothek.
Architektur	Eingebetteter Betrieb	Kein eigener Serverprozess der Bibliothek; keine Abhängigkeit von einem separaten, von der Komponente bereitgestellten Dienst.

Tabelle 9: Übersicht der Systemabhängigkeiten und externen Voraussetzungen

3 Spezifische Anforderungen

Dieses Kapitel bildet den normativen Kern des Dokuments. Funktionale Anforderungen /LF010/ ... /LF250/ und nicht-funktionale /LL010/ ... /LL080/ sind nach SOPHIST/IEEE-830 formuliert. Ergänzt werden externe Schnittstellen (3.3), Produktdaten, Testfälle /TC010/ ... /TC150/ und das Qualitätsmodell nach ISO/IEC 25010 (3.4).

3.1 Funktionale Anforderungen

Das folgende Use-Case-Diagramm ordnet die Anwendungsfälle der öffentlichen API den Akteuren zu. Primärakteur ist die Host-Anwendung; die Sekundärakteure (What3Words-API, Dateisystem, SLF4J-Backend) werden vom System genutzt. Die normative Detailspezifikation je Anwendungsfall folgt anschließend in den /LF.../-Karten.

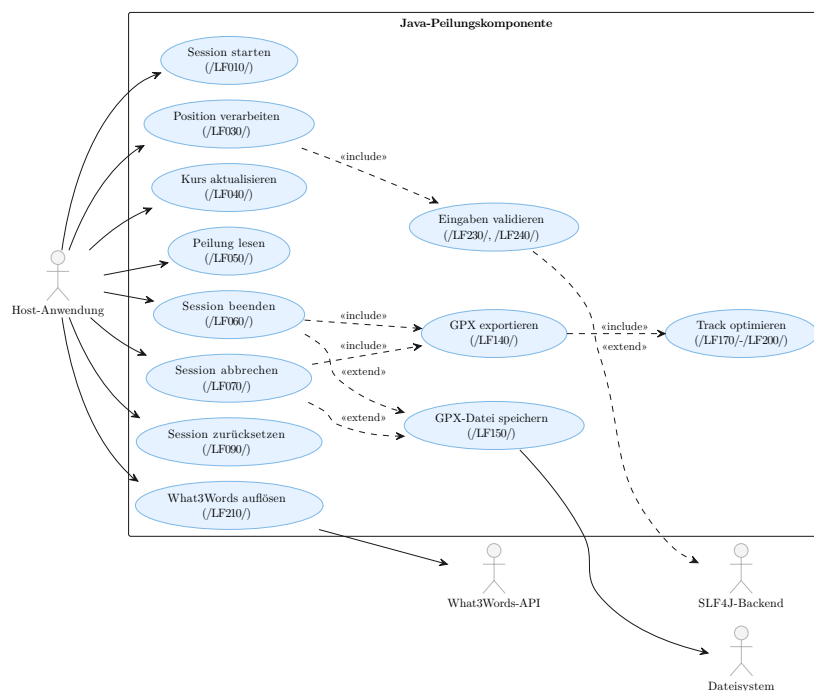


Abbildung 2: Use-Case-Diagramm: Anwendungsfälle der öffentlichen API mit Primär- und Sekundärakteuren.

ID	Funktion	Code-Anker
/LF010/	Session starten	DefaultBearingSession.start
/LF020/	Nur eine aktive Session	start (IDLE/CAS)
/LF030/	Positionsupdate verarbeiten	onPositionUpdate
/LF040/	Kurs aktualisieren	onCourseUpdate
/LF050/	Peilungswerte lesen	currentSnapshot
/LF060/	Session regulär beenden	complete
/LF070/	Session abbrechen	abort
/LF080/	Listener-Fehler isolieren	BearingListener / fire()
/LF090/	Session zurücksetzen	reset

ID	Funktion	Code-Anker
/LF100/	GPS-Rohtrack speichern	TrackAggregator.accept
/LF110/	Soft-Limit warnen	onSoftLimitWarn
/LF120/	Segment bei Zeitlücke	TrackAggregator Split
/LF130/	Hard-Limit Stop	hardLimitPoints (STOP)
/LF140/	GPX 1.1 exportieren	DefaultBearingSession.complete, GpxExportMapper, GpxXmlWriter, OptimizationPipeline
/LF150/	GPX atomar persistieren	SafeFileSink
/LF160/	GPX als UTF-8-Bytes	GpxResult
/LF170/	Optimierung n-ter Punkt	NthPointOptimizer
/LF180/	Optimierung Mindestabstand	MinDistanceOptimizer
/LF190/	Optimierung Geraden-Heuristik	LineCollinearityOptimizer
/LF200/	Optimierung Douglas-Peucker	DouglasPeuckerOptimizer
/LF210/	What3Words auflösen	W3wHttpClient / NoopW3wClient
/LF220/	What3Words cachen	W3wHttpClient LRU/TTL
/LF230/	Koordinaten validieren (GpsFix)	ValidationException COORD_RANGE
/LF240/	Zeitstempel validieren	ValidationException TIMESTAMP_INVALID
/LF250/	Path-Traversal verhindern	SafeFileSink SecurityException

Tabelle 10: Funktionaler Anforderungskatalog /LF010/ ... /LF250/

/ LF010/	Allgemein			
	Funktion	Session starten		
	Akteur	Host-App (Primär).	Verweise	/LF020/, /LF080/, /LD010/, /LD020/
	Include	—	Datenzugriffe	/LD010/, /LD020/
	Beschreibung	Host startet eine Peilungs-Session. System prüft Ziel und Konfiguration, vergibt UUID, friert SessionConfig ein, wechselt zu ACTIVE und benachrichtigt Listener.		
	Funktionsbedingungen			
	Auslöser	Host ruft start(SessionConfig config, GeoCoordinate target) auf.		
	Vorbedingung	Zustand IDLE; gültige GeoCoordinate (WGS84) und SessionConfig.		

	Nachbedingungen (Erfolg)	UUID gesetzt, ACTIVE und onSessionStarted gefeuert.
	Nachbedingungen (Fehler)	/LF020/: IllegalStateException bei Doppelstart. Ungültige GeoCoordinate: IllegalArgumentException bei Erzeugung (Vorbedingung vor start()).
	Funktionsablauf	
	Standardablauf	1. start(config, target) aufrufen. 2. IDLE prüfen (/LF020/). 3. UUID + startedAt, Config einfrieren, ACTIVE. 4. Listener benachrichtigen (/LF080/).
	Alternativablauf	2a Doppelstart: IllegalStateException. 5a Listener wirft: WARN-Log, weiter.
	Sonstiges	Systemgrenzen: Kein GNSS-Hardwarezugriff und kein Netzwerk. , Spezielle Anforderungen: ClockPort-Injection und immutable SessionConfig. , Hinweise: Neustart nach /LF090/.

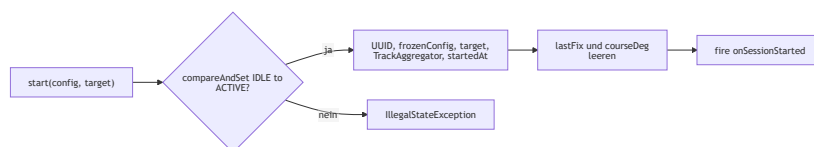


Abbildung 3: Aktivitätsdiagramm /LF010/

/ LF020/	Allgemein			
	Funktion	Nur eine aktive Session		
	Akteur	Host-App (Primär).	Verweise	/LF010/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Zweiter start() im Zustand ACTIVE wird mit IllegalStateException abgewiesen.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		

	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).
	Funktionsablauf	
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.

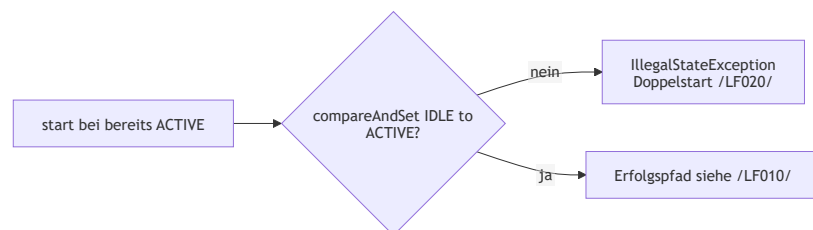


Abbildung 4: Aktivitätsdiagramm /LF020/

/ LF030/	Allgemein			
	Funktion	Positionsupdate verarbeiten		
	Akteur	Host-App (Primär).	Verweise	/LF010/, /LF100/, /LF110/, /LF120/, /LF130/, /LF230/, /LF240/
	Include	/LF100/	Datenzugriffe	/LD040/
	Beschreibung	Host liefert GpsFix. System validiert, speichert im Rohtrack, prüft Limits und Segmente und benachrichtigt Listener. Peilung erfolgt separat via /LF050/.		
	Funktionsbedingungen			
	Auslöser	onPositionUpdate(GpsFix fix)		

	Vorbedingung	Session ACTIVE, Hard-Limit noch nicht erreicht (/LF130/).
	Nachbedingungen (Erfolg)	Fix validiert und gespeichert (Listener informiert).
	Nachbedingungen (Fehler)	/LF230/, /LF240/: ValidationException und /LF130/: kein weiterer Punkt.
	Funktionsablauf	
	Standardablauf	1. Fix validieren (/LF230/, /LF240/). 2. Segment + Budget (/LF120/, /LF110/, /LF130/). 3. Speichern (/LF100/). 4. Listener (onPositionUpdate, ggf. Limit-Events).
	Alternativablauf	Ungültiger Fix: Exception, kein Speichern. Hard-Limit: onHardLimitReached.
	Sonstiges	Spezielle Anforderungen: /LL020/ Azimut-Genauigkeit. , Hinweise: Host steuert Frequenz.

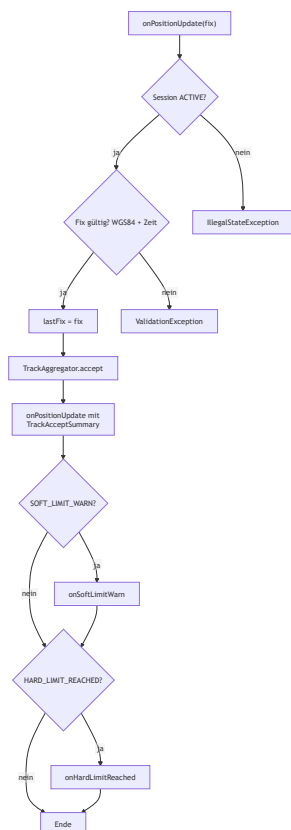


Abbildung 5: Aktivitätsdiagramm /LF030/

	Allgemein
--	------------------

/ LF040/	Funktion	Kurs aktualisieren		
	Akteur	Host-App (Primär).	Verweise	/LF030/, /LF050/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Host setzt geografischen Kurs 0-360° via onCourseUpdateund Wert fließt in /LF050/ ein.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

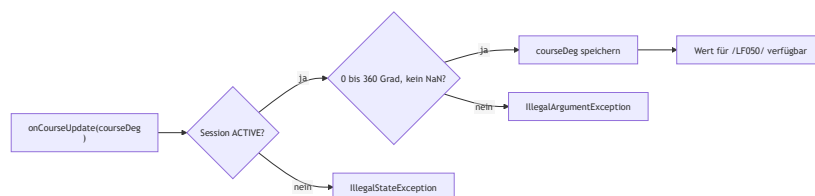


Abbildung 6: Aktivitätsdiagramm /LF040/

/ LF050/	Allgemein			
	Funktion	Peilungswerte lesen		
	Akteur	Host-App (Primär).	Verweise	/LF030/, /LF040/

	Include	—	Datenzugriffe	/LD050/
	Beschreibung	Host fragt currentSnapshot() ab. System liefert Azimut, Distanz, Ordinal (optional Kursabweichung).		
	Funktionsbedingungen			
	Auslöser	currentSnapshot()		
	Vorbedingung	Session ACTIVE, mindestens ein Fix nach /LF030/.		
	Nachbedingungen (Erfolg)	BearingSnapshot zurück, der Zustand bleibt unverändert.		
	Nachbedingungen (Fehler)	Kein Fix: IllegalStateException. Nicht ACTIVE: IllegalStateException.		
	Funktionsablauf			
	Standardablauf	1. currentSnapshot() 2. Snapshot aus letztem Fix + Ziel ableiten 3. Rückgabe		
	Alternativablauf	Kein Fix → IllegalStateException.		
	Sonstiges	Spezielle Anforderungen: Immutables Value Object, 8 Himmelsrichtungen.		

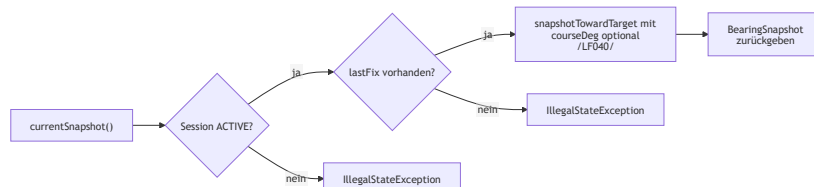


Abbildung 7: Aktivitätsdiagramm /LF050/

/ LF060/	Allgemein			
	Funktion	Session regulär beenden		
	Akteur	Host-App (Primär).	Verweise	/LF140/, /LF150/, /LF160/
	Include	—	Datenzugriffe	/LD060/
	Beschreibung	complete() → COMPLETED, GPX-Export (/LF140/), optional Datei (/LF150/), GpxResult (/LF160/).		
	Funktionsbedingungen			
	Auslöser	complete()		

	Vorbedingung	Session ACTIVE.
	Nachbedingungen (Erfolg)	COMPLETED; GpxResult; onSessionCompleted.
	Nachbedingungen (Fehler)	Nicht ACTIVE: <code>IllegalStateException</code> . Path Traversal: <code>/LF250/</code> .
	Funktionsablauf	
	Standardablauf	1. <code>complete()</code> 2. Segment schließen 3. GPX (<code>/LF140/</code>) 4. optional <code>persist</code> (<code>/LF150/</code>) 5. Listener
	Alternativablauf	IO optiona, GPX-Bytes dennoch in <code>GpxResult</code> .
	Sonstiges	Hinweise: Danach <code>/LF090/</code> für Neustart.

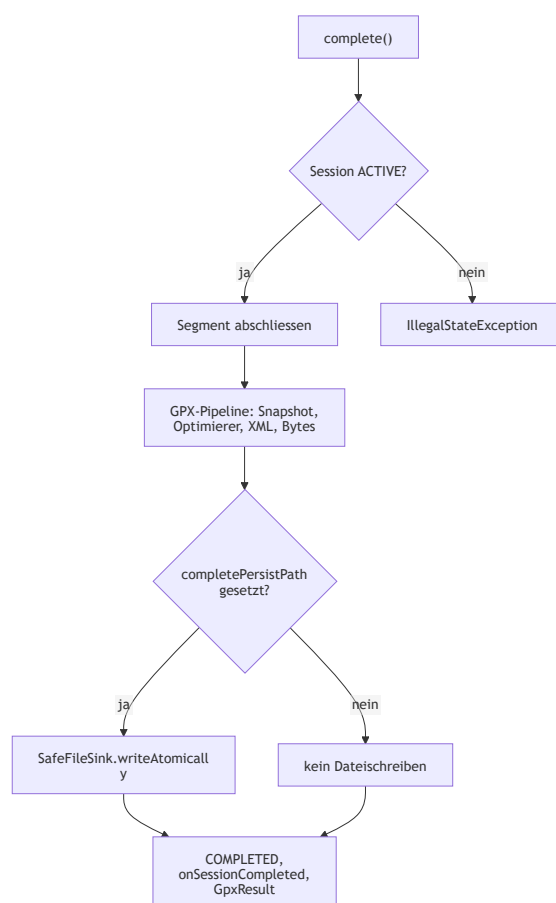


Abbildung 8: Aktivitätsdiagramm /LF060/

/ LF070/	Allgemein	
	Funktion	Session abbrechen

	Akteur	Host-App (Primär).	Verweise	/LF140/, /LF160/
	Include	—	Datenzugriffe	—
	Beschreibung	abort() → ABORTED, GPX mit bisherigem Track, Datei nur bei persistOnAbort.		
	Funktionsbedingungen			
	Auslöser	abort()		
	Vorbedingung	Session ACTIVE.		
	Nachbedingungen (Erfolg)	ABORTED, GpxResult, onSessionAborted.		
	Nachbedingungen (Fehler)	Nicht ACTIVE: IllegalStateException.		
	Funktionsablauf			
	Standardablauf	1. abort() 2. GPX erzeugen 3. optional Datei 4. Listener		
	Alternativablauf	Leerer Track: valides GPX mit leerem trk.		
	Sonstiges	Hinweise: Track wird nicht verworfen.		

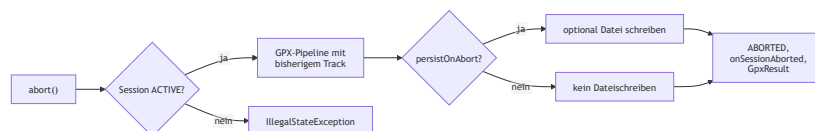


Abbildung 9: Aktivitätsdiagramm /LF070/

/ LF080/	Allgemein			
	Funktion	Listener-Fehler isolieren		
	Akteur	Host-App (fehlerhafter Listener).	Verweise	/LF010/, /LF030/, /LF060/, /LF070/
	Include	—	Datenzugriffe	—
	Beschreibung	Wirft ein BearingListener bei Callback, wird die Exception geloggt und übrige Listener werden weiter aufgerufen.		
	Funktionsbedingungen			
	Auslöser	Listener-Callback während Session-Events.		

	Vorbedingung	Listener registriert via <code>addListener</code> .
	Nachbedingungen (Erfolg)	Session konsistent und restliche Listener benachrichtigt.
	Nachbedingungen (Fehler)	-
	Funktionsablauf	
	Standardablauf	1. Callback 2. Exception fangen 3. WARN-Log mit sessionId 4. nächster Listener
	Alternativablauf	-
	Sonstiges	Spezielle Anforderungen: /LL060/ Logging.

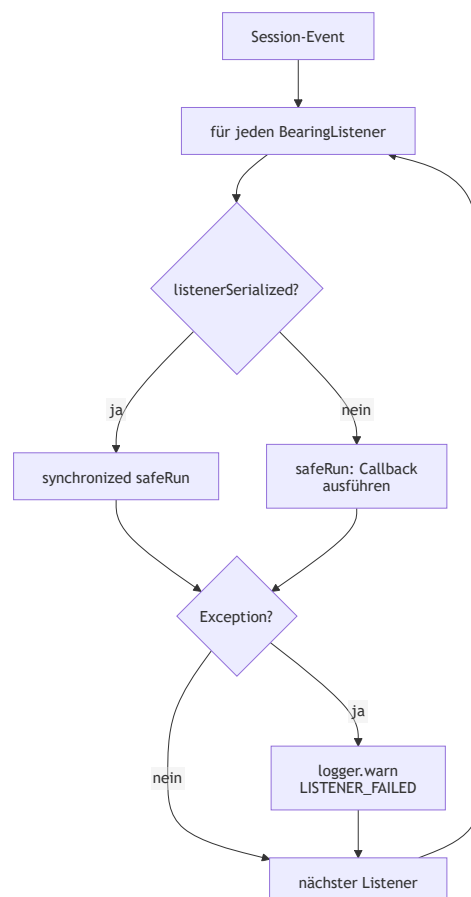


Abbildung 10: Aktivitätsdiagramm /LF080/

/ LF090/	Allgemein	
	Funktion	Session zurücksetzen

	Akteur	Host-App (Primär).	Verweise	/LF010/, /LF060/, /LF070/
	Include	—	Datenzugriffe	—
	Beschreibung	Nach COMPLETED/ABORTED setzt reset() auf IDLE, erneuter /LF010/ ohne JVM-Neustart.		
	Funktionsbedingungen			
	Auslöser	reset()		
	Vorbedingung	Zustand COMPLETED oder ABORTED.		
	Nachbedingungen (Erfolg)	IDLE; Felder gelöscht; /LF010/ wieder möglich.		
	Nachbedingungen (Fehler)	Bei ACTIVE: IllegalStateException.		
	Funktionsablauf			
	Standardablauf	1. reset() 2. Zustand prüfen 3. Felder löschen 4. IDLE		
	Alternativablauf	ACTIVE → Exception.		
	Sonstiges	Hinweise: /TC130/ Verifikation.		

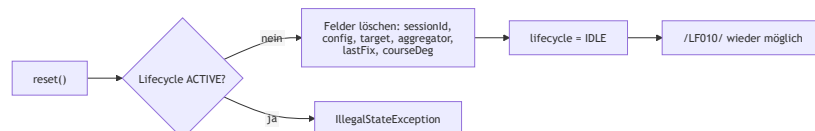


Abbildung 11: Aktivitätsdiagramm /LF090/

/ LF100/	Allgemein			
	Funktion	GPS-Rohtrack speichern		
	Akteur	Host-App (Primär).	Verweise	/LF030/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Jeder validierte Fix wird im Rohspeicher abgelegt (/LD030/).		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		

	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.

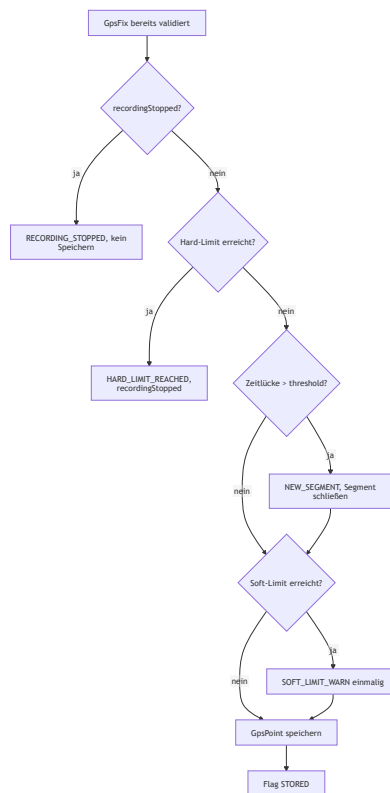


Abbildung 12: Aktivitätsdiagramm /LF100/

/ LF110/	Allgemein			
	Funktion	Soft-Limit warnen		
	Akteur	Host-App (Primär).	Verweise	/LF030/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Bei Erreichen des Soft-Limits einmalig onSoftLimitWarn (Aufzeich- nung läuft weiter).		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain- Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

/ LF120/	Allgemein			
	Funktion	Segment bei Zeitlücke		
	Akteur	Host-App (Primär).	Verweise	/LF030/, /LF100/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Zeitlücke > segmentGapThreshold eröffnet neues trkseg (/TC150/).		
	Funktionsbedingungen			

	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.
	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (<code>ValidationException</code> , <code>IllegalStateException</code> , <code>SecurityException</code>).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende <code>ValidationException</code> oder <code>IllegalStateException</code> .
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: <code>#code</code> . , Hinweise: Normiert in <code>requirement-catalog.typ</code> .

/ LF130/	Allgemein			
	Funktion	Hard-Limit Stop		
	Akteur	Host-App (Primär).	Verweise	/LF030/, /LF100/
	Include	—	Datenzugriffe	—
	Beschreibung	Bei Hard-Limit stoppt die Aufzeichnung (overflowMode.STOP), onHardLimitReached. Kein Downsampling.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.		

	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (<code>ValidationException</code> , <code>IllegalStateException</code> , <code>SecurityException</code>).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende <code>ValidationException</code> oder <code>IllegalStateException</code> .
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: <code>#code</code> . , Hinweise: Normiert in <code>requirement-catalog.typ</code> .

/ LF140/	Allgemein			
	Funktion	GPX 1.1 exportieren		
	Akteur	System (intern via / LF060/, /LF070/).	Verweise	/LF170/-/LF200/, / LF160/, /LL050/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Rohtrack-Snapshot → optionale Optimierer-Kette → GPX 1.1 XML mit Namespace und UTC-Zeiten.		
	Funktionsbedingungen			
	Auslöser	complete() oder abort().		
	Vorbedin- gung	Session beendet oder Export intern angestoßen.		
	Nachbedin- gungen (Er- folg)	GpxDocument serialisiert (/LF160/ erfüllt).		
	Nachbedin- gungen (Feh- ler)	Serialisierungsfehler: RuntimeException aus GpxXmlWriter.		
	Funktionsablauf			
	Standardab- lauf	1. Immutable Track-Snapshot. 2. Optimierer (/LF170/-/LF200/) falls konfiguriert. 3. GpxExportMapper + GpxXmlWriter (/LL050/ Escaping).		
	Alternativ- ablauf	Keine Optimierer: Rohpunkte exportieren.		

	Sonstiges	Spezielle Anforderungen: GPX 1.1 Namespace, UTF-8 ohne BOM.
--	-----------	--

/ LF150/	Allgemein			
	Funktion	GPX atomar persistieren		
	Akteur	Host-App (Primär).	Verweise	/LF060/, /LF070/, /LF250/
	Include	—	Datenzugriffe	—
	Beschreibung	Optional atomares Schreiben via SafeFileSink bei konfiguriertem Pfad.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

/ LF160/	Allgemein			
	Funktion	GPX als UTF-8-Bytes		
	Akteur	Host-App (Primär).	Verweise	/LF140/

	Include	—	Datenzu- griffe	—
	Beschrei- bung	GpxResult liefert utf8Bytes() und asUtf8String() ohne BOM.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

/ LF170/	Allgemein			
	Funktion	Optimierung n-ter Punkt		
	Akteur	Host-App (Primär).	Verweise	/LF140/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Strategy: jeder n-te Punkt, Start/Ende erhalten.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		

	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.

/ LF180/	Allgemein			
	Funktion	Optimierung Mindestabstand		
	Akteur	Host-App (Primär).	Verweise	/LF140/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Punkte mit Mindestabstand in Metern.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			

	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende <code>ValidationException</code> oder <code>IllegalStateException</code> .
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: <code>#code</code> . , Hinweise: Normiert in <code>requirement-catalog.typ</code> .

/ LF190/	Allgemein			
	Funktion	Optimierung Geraden-Heuristik		
	Akteur	Host-App (Primär).	Verweise	/LF140/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Kollinearität reduziert Zwischenpunkte.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code.		

		, Hinweise: Normiert in requirement-catalog.typ.
--	--	--

/ LF200/	Allgemein			
	Funktion	Optimierung Douglas-Peucker		
	Akteur	Host-App (Primär).	Verweise	/LF140/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Douglas-Peucker mit metrischem Epsilon.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

/ LF210/	Allgemein			
	Funktion	What3Words auflösen		
	Akteur	Host-App (Primär).	Verweise	/LF220/
	Include	—	Datenzugriffe	—

	Beschreibung	Reverse-Lookup via injiziertem <code>W3wClientPort</code> und Fallback bei Offline (<code>NoopW3wClient</code>).
	Funktionsbedingungen	
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.
	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (<code>ValidationException</code> , <code>IllegalStateException</code> , <code>SecurityException</code>).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende <code>ValidationException</code> oder <code>IllegalStateException</code> .
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: <code>#code</code> . , Hinweise: Normiert in <code>requirement-catalog.typ</code> .

/ LF220/	Allgemein			
	Funktion	What3Words cachén		
	Akteur	Host-App (Primär).	Verweise	/LF210/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	LRU-Cache mit TTL im W3wHttpClient.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		

	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).
	Funktionsablauf	
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.

/ LF230/	Allgemein			
	Funktion	Koordinaten validieren		
	Akteur	Host-App (Primär).	Verweise	/LF030/
	Include	—	Datenzugriffe	—
	Beschreibung	WGS84-Bereich bei GpsFix in /LF030/, ValidationException(COORD_RANGE).		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedingung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedingungen (Erfolg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedingungen (Fehler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardablauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf.		

		2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.
	Alternativablauf	Ungültige Eingabe: passende <code>ValidationException</code> oder <code>IllegalStateException</code> .
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: <code>#code</code> . , Hinweise: Normiert in <code>requirement-catalog.typ</code> .

/ LF240/	Allgemein			
	Funktion	Zeitstempel validieren		
	Akteur	Host-App (Primär).	Verweise	/LF030/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Zukunft >1s oder älter als maxFixAge; TIMESTAMP_INVALID.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

/ LF250/	Allgemein			
	Funktion	Path-Traversal verhindern		
	Akteur	Host-App (Primär).	Verweise	/LF150/
	Include	—	Datenzu- griffe	—
	Beschrei- bung	Pfade nur unter allowedBaseDir; sonst SecurityException.		
	Funktionsbedingungen			
	Auslöser	Siehe Standardablauf; Aufruf über Session-API oder Domain-Service.		
	Vorbedin- gung	Session ACTIVE sofern Session-bezogen, sonst keine.		
	Nachbedin- gungen (Er- folg)	Anforderung erfüllt, kein undefinierter Zustand.		
	Nachbedin- gungen (Feh- ler)	Fachliche Ausnahme gemäß API (ValidationException, IllegalStateException, SecurityException).		
	Funktionsablauf			
	Standardab- lauf	1. Host oder Export-Pipeline ruft die zugehörige Komponente auf. 2. System führt die in der Beschreibung genannte Logik aus. 3. Ergebnis wird zurückgegeben oder persistiert.		
	Alternativ- ablauf	Ungültige Eingabe: passende ValidationException oder IllegalStateException.		
	Sonstiges	Systemgrenzen: Kein UI und kein direkter Hardwarezugriff. , Spezielle Anforderungen: Implementierung: #code. , Hinweise: Normiert in requirement-catalog.typ.		

3.2 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben **wie** das System seine Aufgaben erfüllt (/LL010/ ... /LL080/).

ID	Nicht-funktionale Anforderung
/LL010/	Eingabevalidierung - kritische Fehlerpfade getestet
/LL020/	Azimut $\pm 1^\circ$ vs. Haversine-Referenz
/LL030/	Portabilität Java 11+, keine UI-Frameworks
/LL040/	<code>mvn test</code> reproduzierbar auf frischem Clone
/LL050/	XML-Escaping in GPX-Ausgabe
/LL060/	Strukturiertes Logging mit <code>sessionId</code> in der Log-Nachricht
/LL070/	Test-Determinismus via <code>ClockPort-Mock</code>
/LL080/	Öffentliche API mit Javadoc (ohne Build-Gate)

Tabelle 11: Nicht-funktionale Anforderungen /LL010/ ... /LL080/.

/ LL010/	Funktion	Eingabevalidierung		
	Quelle	Qualitätsrichtlinie	Verweise	/LF030/, /LF230/, /LF240/
	Beschreibung	Kritische Fehlerpfade (ungültige Koordinaten, Zeitstempel) erzeugen definierte <code>ValidationException</code> und sind durch /TC020/, /TC030/ abgesichert.		

/ LL020/	Funktion	Genauigkeit der Peilungsberechnung		
	Quelle	Fachanforderung	Verweise	/LF030/, /LF050/
	Beschreibung	Azimut-Abweichung $\leq \pm 1^\circ$ vs. Haversine-Referenz für $D > 10$ m; Nachweis /TC140/.		

/ LL030/	Funktion	Portabilität		
	Quelle	Projektauftrag	Verweise	-
	Beschreibung	Java 11+, keine UI-Framework-Abhängigkeiten, plattformunabhängige Pfade und APIs.		

/ LL040/	Funktion	Build-Reproduzierbarkeit		
	Quelle	Projektauftrag	Verweise	-
	Beschreibung	<code>mvn clean test</code> auf frisch geklontem Repository erfolgreich und Abhängigkeiten über Maven Central.		

/ LL050/	Funktion	XML-Escaping		
	Quelle	Sicherheitsrichtlinie	Verweise	/LF140/
	Beschreibung	Nutzerstrings in GPX werden escaped (<code>XmlEscaper</code>) und Nachweis über /TC100/ und GPX-Writer-Tests.		

/ LL060/	Funktion	Strukturiertes Logging		
	Quelle	Betriebsanforderung	Verweise	/LF080/
	Beschreibung	WARN/ERROR via SLF4J, sessionId in der Log-Nachricht (Slf4jLoggerAdapter).		

/ LL070/	Funktion	Test-Determinismus		
	Quelle	Testbarkeitsanforderung	Verweise	-
	Beschreibung	Zeitabhängige Tests nutzen ClockPort-Mocks und kein Thread.sleep() im Testcode.		

/ LL080/	Funktion	Javadoc der öffentlichen API		
	Quelle	Vorlesung SWE	Verweise	-
	Beschreibung	Öffentliche API-Klassen und -Methoden sind mit Javadoc dokumentiert (ohne Build-Gate).		

3.3 Externe Schnittstellen

Die Bibliothek besitzt keine Benutzeroberfläche; ihre einzige Schnittstelle nach außen ist die öffentliche Java-API. Dieser Abschnitt beschreibt die aufrufbaren Methoden, das Fehlverhalten und das Format der erzeugten GPX-Ausgabe. Zusammen bilden sie den Vertrag, auf den sich eine Host-Anwendung verlassen kann.

Das Kontextdiagramm (Datenflussdiagramm der Stufe 0) zeigt die Bibliothek als einen Prozess mit ihren externen Terminatoren und den ein- und ausgehenden Datenflüssen.

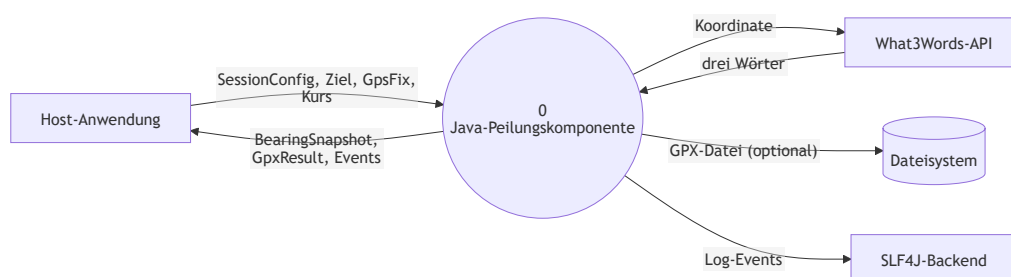


Abbildung 13: Kontextdiagramm (DFD Stufe 0): Datenflüsse zwischen Bibliothek und Umgebung.

3.3.1 Öffentliche Java-API

Alle Aufrufe laufen über eine einzige Fassade. Sie spiegelt den Session-Lebenszyklus wider: Eine Session wird gestartet, mit Positions- und Kursdaten versorgt, abgefragt und schließlich regulär beendet oder abgebrochen. Jede Methode ist einer funktionalen Anforderung zugeordnet, was die Spalte *LF* festhält.

Signatur	Beschreibung	LF
<code>void start(SessionConfig, GeoCoordinate)</code>	Session starten	/ LF010/
<code>void onPositionUpdate(GpsFix)</code>	GPS-Fix verarbeiten	/ LF030/
<code>void onCourseUpdate(double)</code>	Kurs 0-360°	/ LF040/
<code>BearingSnapshot currentSnapshot()</code>	Peilungsschnappschuss	/ LF050/
<code>GpxResult complete()</code>	Regulär beenden	/ LF060/
<code>GpxResult abort()</code>	Abbrechen	/ LF070/
<code>void reset()</code>	Auf IDLE zurücksetzen	/ LF090/
<code>Optional<String> resolveWhat3Words(lat, lon)</code>	W3W optional	/ LF210/
<code>void addListener(BearingListener)</code>	Listener (DefaultBearingSession)	-

Tabelle 12: Öffentliche API-Methoden der Bibliothek (`BearingSession` / `DefaultBearingSession`).

Die beiden folgenden Sequenzdiagramme zeigen die dynamische Zusammenarbeit hinter den zentralen API-Aufrufen: das Verarbeiten eines Positionsupdates und den regulären Abschluss mit GPX-Export.

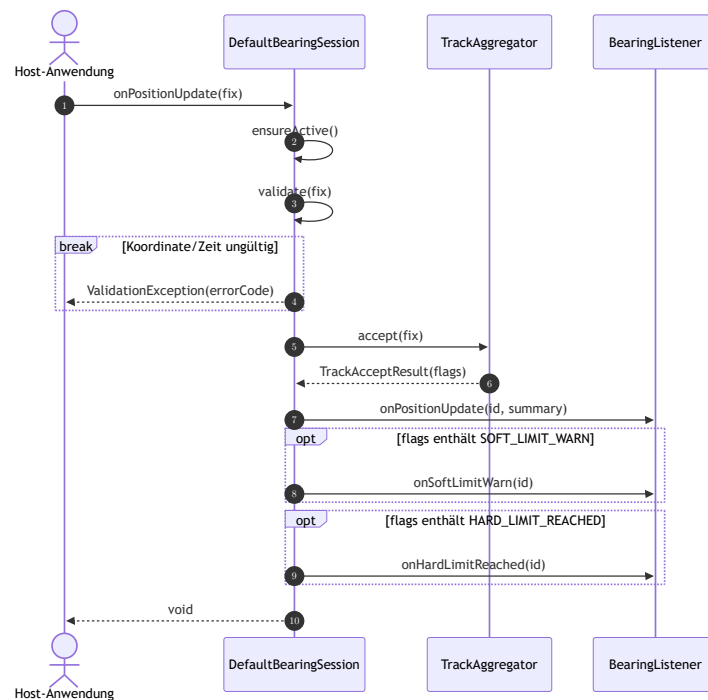


Abbildung 14: Sequenzdiagramm `onPositionUpdate()`: Validierung, Aggregation und Listener-Ereignisse (/LF030/).

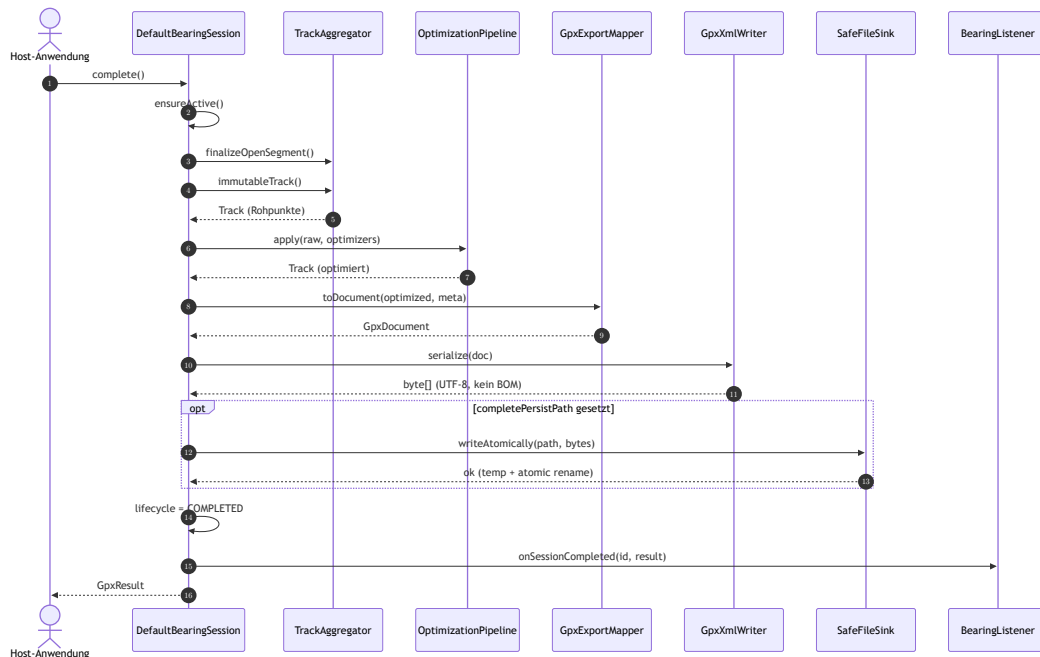


Abbildung 15: Sequenzdiagramm `complete()`: Export-Kette von der Aggregation bis zum `GpxResult` (/LF060/, /LF140/).

3.3.2 Fehlerverhalten

Die Bibliothek nutzt durchgängig **unchecked** Exceptions (`RuntimeException` und Subtypen). Der Host muss sie nicht deklarieren, kann sie aber gezielt abfangen. Details je Anforderung stehen in den /LF.../-Karten, hier die Zuordnung nach Fehlertyp:

Exception	Typische Auslöser	Anforderungen
ValidationException	Ungültige Koordinaten oder Zeitstempel bei onPositionUpdate(GpsFix). Feld errorCode(): COORD_RANGE oder TIMESTAMP_INVALID.	/LF230/, / LF240/ (via / LF030/)
IllegalStateException	Session nicht ACTIVE (z. B. Update oder Snapshot vor Start, nach complete()/abort()). Doppelstart: reset() während ACTIVE, Snapshot ohne vorherigen Fix.	/LF020/, / LF030/, / LF050/, / LF060/, / LF070/, / LF090/
IllegalArgumentException	Kurs außerhalb 0-360° (onCourseUpdate), ungültige GeoCoordinate bei Erzeugung und fehlerhafte SessionConfig beim build().	/LF040/, / LF010/ (Ziel), / LD020/
SecurityException	Persistenzpfad liegt außerhalb von allowedBaseDir.	/LF250/ (via / LF150/)
RuntimeException	I/O-Fehler beim atomaren Dateischreiben (SafeFileSink).	/LF150/

Tabelle 13: Geworfene Exceptions nach Auslöser (Vertrag der öffentlichen API).

Die GPX-Serialisierung (GpxXmlWriter, /LF140/) definiert unter normalen Eingaben keine eigenen Fehlertypen. Listener-Fehler (/LF080/) werden intern abgefangen, die Session bleibt konsistent.

3.3.3 GPX 1.1-Ausgabeformat

Der Export folgt dem GPX-1.1-Schema von Topografix (/LF140/, /LF160/). Verbindlich sind Namespace, UTF-8 ohne BOM und UTC-Zeitstempel (z). Metadaten enthalten Name und Zeitspanne (<bounds> ist optional).

GPX-1.1-Schemastruktur (normativ)

```
<?xml version="1.0" encoding="UTF-8"?>
<gpx version="1.1" creator="Java-Kompass/1.0"
  xmlns="http://www.topografix.com/GPX/1/1"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.topografix.com/GPX/1/1
    http://www.topografix.com/GPX/1/1/gpx.xsd">

  <metadata>
    <name>Peilung Stuttgart - Schlossplatz</name>
    <time>2026-06-12T10:00:00Z</time>
  </metadata>
  <trk>
    <trkseg>
      <trkpt lat="48.7758" lon="9.1829">
        <ele>248.0</ele>
        <time>2026-06-12T10:01:00Z</time>
        <hdop>1.2</hdop>
      </trkpt>
    </trkseg>
  </trk>
</gpx>
```

```

    </trkpt>
  </trkseg>
</trk>
</gpx>

```

3.4 Qualitäts-Anforderungen

3.4.1 Qualitätsmodell nach ISO/IEC 25010

ISO/IEC 25010 beschreibt Produktqualität anhand von acht Hauptmerkmalen mit ihren Unterkriterien. Die folgende Matrix gewichtet jedes Unterkriterium nach seiner Bedeutung für genau dieses Projekt - von *sehr wichtig* bis *nicht relevant*. Die Gewichtung macht die Schwerpunkte sichtbar: Hoch eingestuft sind vor allem die Richtigkeit der Berechnung, die Zuverlässigkeit und die Wartbarkeit, während Aspekte wie Verbrauchsverhalten oder Anpassbarkeit für eine eingebettete Bibliothek dieser Größe nachrangig sind.

Produktqualität	sehr wichtig	wichtig	normal	nicht relevant
Funktionalität				
Angemessenheit	x			
Sicherheit			x	
Interoperabilität			x	
Konformität		x		
Ordnungsmäßigkeit			x	
Richtigkeit		x		
Zuverlässigkeit				
Fehlertoleranz			x	
Konformität			x	
Reife		x		
Wiederherstellbarkeit		x		
Benutzbarkeit				
Attraktivität	x			
Bedienbarkeit		x		
Erlernbarkeit	x			
Konformität			x	
Verständlichkeit		x		
Effizienz				
Konformität			x	
Zeitverhalten			x	
Verbrauchsverhalten				x
Änderbarkeit				
Analysierbarkeit			x	
Konformität			x	

Modifizierbarkeit		X		
Stabilität		X		
Testbarkeit			X	
Übertragbarkeit				
Anpassbarkeit				X
Austauschbarkeit		X		
Installierbarkeit			X	
Koexistenz		X		
Konformität			X	

Tabelle 14: Gewichtung der Qualitätsmerkmale nach ISO/IEC 25010 (Relevanzstufen je Unterkriterium).

3.4.2 Produktdaten (/LD.../)

Die Produktdaten beschreiben die zentralen Datenstrukturen der Bibliothek mit ihren Feldern, Typen und Wertebereichen. Sie bilden die Brücke zwischen den funktionalen Anforderungen und der späteren Implementierung: Jede /LD.../-Karte verweist auf die Anforderungen, in denen die Struktur entsteht oder verwendet wird.

Das ER-Modell stellt diese Datenstrukturen mit ihren Beziehungen und Kardinalitäten dar und ordnet sie den /LD.../-Karten zu.

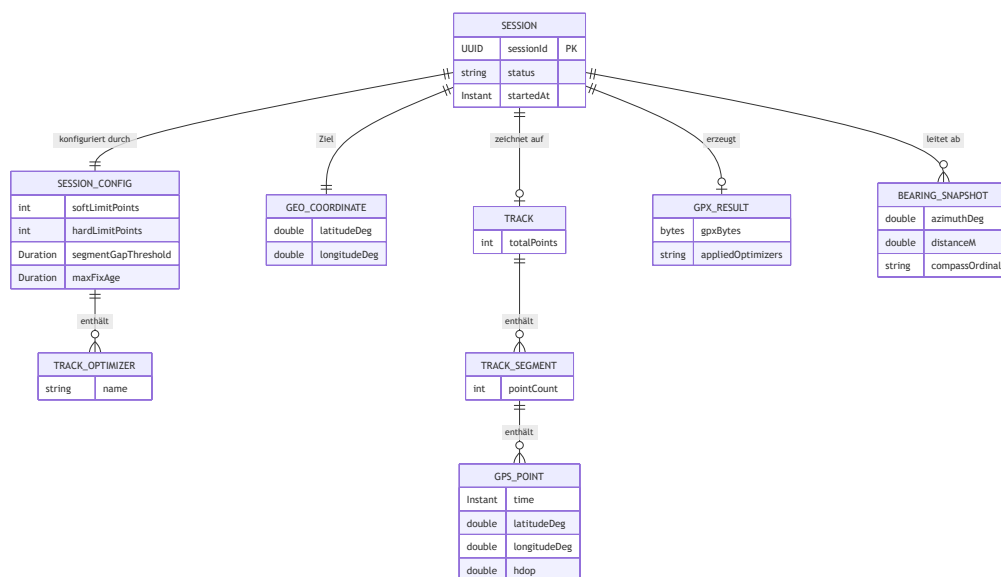


Abbildung 16: ER-Modell der Produktdaten (/LD010/ ... /LD060/) mit Entitäten und Kardinalitäten.

ID	Produktdaten
/LD010/	Peilungs-Session
/LD020/	SessionConfig
/LD030/	GPS-Track / Segment
/LD040/	GpsPoint / GpsFix
/LD050/	BearingSnapshot
/LD060/	GpxResult

Tabelle 15: Produktdaten /LD010/ ... /LD060/.

/LD010/	Speicherinhalt	Peilungs-Session	
	Verweise	/LF010/, /LF060/, /LF070/, /LF090/	
	Attribute	sessionId	UUID RFC 4122 v4
		status	IDLE · ACTIVE · COMPLETED · ABORTED
		target	GeoCoordinate (WGS84)
		startedAt	Instant UTC
		config	SessionConfig (eingefroren)

/LD020/	Speicherinhalt	SessionConfig	
	Verweise	/LF010/, /LF110/, /LF130/, /LF120/	
	Attribute	softLimitPoints	int ≥ 0
		hardLimitPoints	int ≥ 0; Hard-Limit /LF130/ STOP
		segmentGapThreshold	Duration (Standard: PT5M)
		maxFixAge	Duration (Standard: PT24H)
		persistOnAbort	boolean
		optimizers	List<TrackOptimizer> (/LF170/-/LF200/)

/LD030/	Speicherinhalt	GPS-Track / Segment	
	Verweise	/LF100/, /LF120/, /LF140/	
	Attribute	segments	List<TrackSegment>
		totalPoints	int (abgeleitet)

/LD040/	Speicherinhalt	GpsPoint / GpsFix	
	Verweise	/LF030/, /LF230/, /LF240/	
	Attribute	lat	double $\in [-90,0, +90,0]$
		lon	double $\in [-180,0, +180,0]$
		time	Instant UTC (Pflichtfeld)
		ele	Optional<Double> (Höhe in m)
		hdop	Optional<Double> (≥ 0)
		speed	Optional<Double> (≥ 0 m/s)

/LD050/	Speicherinhalt	BearingSnapshot	
	Verweise	/LF050/	
	Attribute	azimuthDeg	double $\in [0,0^\circ, 360,0^\circ]$, [distanceM], [double ≥ 0], [compassOrdinal], [Enum: N · NE · E · SE · S · SW · W · NW], [bearingErrorDeg], [Optional<Double> $\in (-180^\circ, +180^\circ]$

/LD060/	Speicherinhalt	GpxResult	
	Verweise	/LF140/, /LF160/	
	Attribute	gpxBytes	byte[] UTF-8, kein BOM
		statistics	SessionStatistics
		appliedOptimizers	List<String>

3.4.3 Testfälle (Traceability)

Verbindliche Testfälle /TC010/ ... /TC150/ mit JUnit-Methodennamen (siehe implementation/TRACEABILITY.md). Ergänzende Nachweise ohne /TC.../ (z. B. /LF150/) stehen dort im gleichen Dokument.

TC	LF / LL	Beschreibung	JUnit
/TC010/	/LF010/, /LF020/	Start + Doppelstart	tc010_doubleStartThrows
/TC020/	/LF030/, /LF230/	Ungültige Breite	tc020_invalidLatThrows

TC	LF / LL	Beschreibung	JUnit
/TC030/	/LF030/, /LF240/	Zeitstempel Zukunft	tc030_futureTimestampThrows
/TC040/	/LF060/, /LF030/	Update nach complete	tc040_updateAfterCompleteThrows
/TC050/	/LF050/	Snapshot ohne Fix	tc050_snapshotWithoutFixThrows
/TC060/	/LF110/	Soft-Limit	tc060_softLimitWarns
/TC070/	/LF130/	Hard-Limit Stop	tc070_hardLimitStopsRecording
/TC080/	/LF070/	abort liefert GPX	tc080_abortProducesGpx
/TC090/	/LF250/	Path Traversal	tc090_pathTraversalSecurity
/TC100/	/LF140/	GPX-Namespace 1.1	tc100_gpxNamespace
/TC110/	/LF170/	n-Optimizer	tc110_nthOptimizer
/TC120/	/LF080/	Listener isoliert	tc120_listenerExceptionIsolated
/TC130/	/LF090/	reset + Neustart	tc130_resetAllowsRestart
/TC140/	/LL020/	Azimut-Referenz	tc140_azimuthReference
/TC150/	/LF120/	Segment bei Zeitlücke	tc150_segmentSplit

Tabelle 16: Testfälle mit Traceability zu /LF.../ und /LL.../.

3.4.4 FMEA

Die Fehlermöglichkeits- und Einflussanalyse (FMEA) betrachtet systematisch, was schiefgehen kann, welche Folge das hätte und wie die Bibliothek dem begegnet. Jede Zeile schätzt Auftretenswahrscheinlichkeit (A) und Entdeckungswahrscheinlichkeit bzw. Schwere der Folge (E) qualitativ ein und nennt die konkrete Gegenmaßnahme samt zugehöriger Anforderung.

Fehler	Folge	A	E	Gegenmaßnahme
Ungültige Koordinate	Falsche Peilung oder Exception.	M	K	ValidationException, /LF230/.
Ungültiger Zeitstempel	Falsche Segmentierung.	M	M	/LF240/.
Speicheroverflow	OutOfMemoryError der JVM.	G	K	Hard-Limit /LF130/.
XML-Injection	Ungültige GPX-Datei.	G	M	XmlEscaper, /LF140/, /LL050/.
Path Traversal	Kritische Datei überschrieben.	G	M	/LF250/.
W3W-Ausfall	Fehlende W3W-Adresse.	G	G	Fallback, /LF210/, /LF220/.

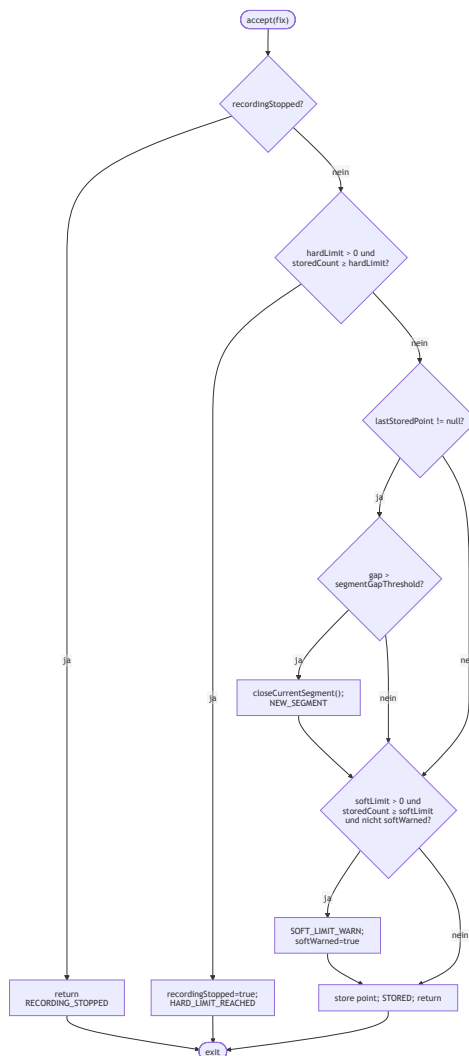
Fehler	Folge	A	E	Gegenmaßnahme
Listener wirft	Benachrichtigungsschleife abbricht.	M	K	/LF080/, /TC120/.
Clock nicht injiziert	Nicht-deterministische Tests.	S	G	clockPort, /LL070/.

Tabelle 17: FMEA: Fehlermodi, Folgen, Klassen und Gegenmaßnahmen.

Legende A/E: S = selten, M = mittel, G = gering, K = katastrophal (qualitativ).

3.4.5 Whitebox-Sicht: Kontrollfluss und Entscheidungslogik

Ergänzend zur Testfall-Tabelle zeigt dieser Abschnitt die Kontroll- und Entscheidungslogik der Eingabeverarbeitung. Der Kontrollflussgraph der Methode `TrackAggregator.accept` besitzt fünf Entscheidungen und damit die zyklomatische Komplexität $v(G) = 5 + 1 = 6$; sechs unabhängige Pfade bilden das Mindestmaß für die Zweigüberdeckung (/TC060/, /TC070/, /TC110/, /TC150/).


 Abbildung 17: Kontrollflussgraph von `TrackAggregator.accept` (Whitebox-Sicht, $v(G) = 6$).

Die Eingabevalidierung von `onPositionUpdate` lässt sich gleichwertig als Entscheidungstabelle und als Entscheidungsbaum darstellen. Beide bilden dieselbe Regelmenge ab und machen die Vollständigkeit der Fehlerpfade prüfbar (/LL010/, /TC020/, /TC030/).

	R1	R2	R3	R4	R5
Session ACTIVE?	N	J	J	J	J
Koordinate WGS84-gültig?	-	N	J	J	J
Zeitstempel gültig?	-	-	N	J	J
Hard-Limit erreicht?	-	-	-	J	N
IllegalStateException	X				
ValidationException COORD		X			
ValidationException TIME			X		
onHardLimitReached / STOP				X	
Fix speichern (+ Events)					X

Abbildung 18: Entscheidungstabelle der Fix-Verarbeitung (Bedingungen und Regeln R1–R5).

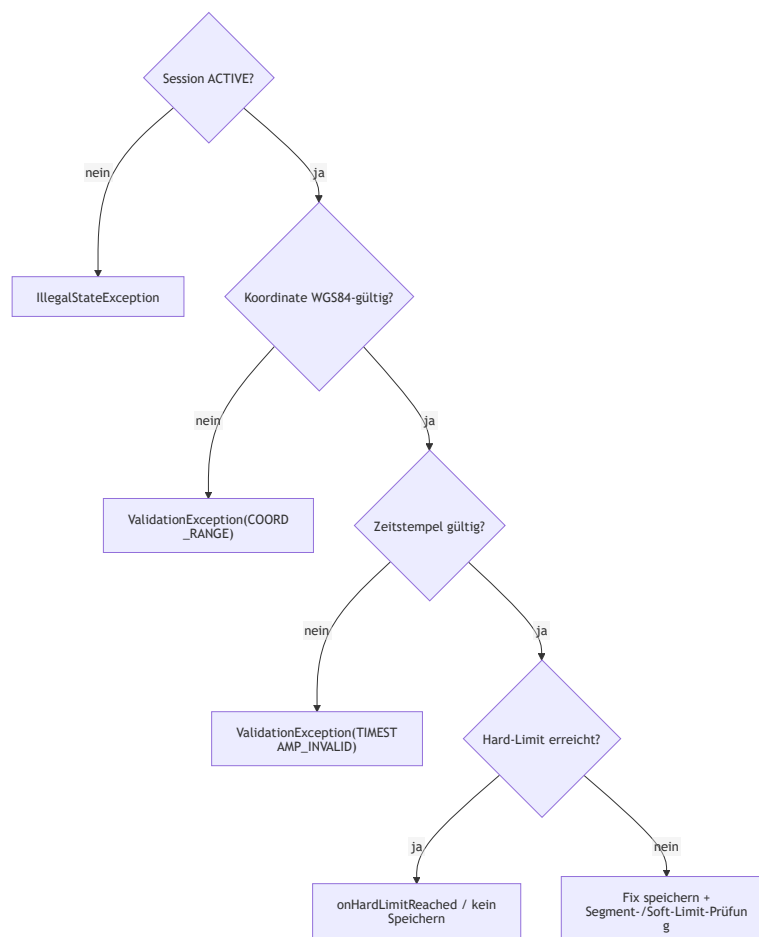


Abbildung 19: Entscheidungsbaum der Fix-Verarbeitung (Baumform der Entscheidungstabelle).

4 Anhang

Der Anhang ergänzt den normativen Teil um vertiefende Artefakte: die vollständige Traceability-Matrix, das objektorientierte Analyse- und Entwurfsmodell, den Referenzalgorithmus der Peilung sowie Literatur- und Normenverweise.

4.1 Vollständige Traceability-Matrix

Die folgende Matrix stellt die vollständige Rückverfolgbarkeit von Anforderungen über OOA-Konzepte und OOD-Klassen bis zu Testclustern her. Die Testzuordnung zu /TC.../ ist in Kap. 3.4 detailliert.

/LF	OOA-Konzept	OOD-Klasse	Test
/LF010/	Peilung	DefaultBearingSession	/TC010/
/LF020/	Peilung	DefaultBearingSession	/TC010/
/LF030/	Standortfolge	DefaultBearingSession, GpsFix	/TC020/, /TC030/, /TC040/
/LF040/	Peilung	DefaultBearingSession	implizit Demo
/LF050/	Peilungsgrößen	BearingSnapshot	/TC050/
/LF060/	Export	DefaultBearingSession	/TC040/
/LF070/	Peilung	DefaultBearingSession	/TC080/
/LF080/	Peilung	BearingListener	/TC120/
/LF090/	Peilung	DefaultBearingSession	/TC130/
/LF100/	Standortfolge	TrackAggregator	implizit /TC060/, /TC070/, / TC110/, /TC150/
/LF110/	Standortfolge	TrackAggregator	/TC060/
/LF120/	Track	TrackAggregator	/TC150/
/LF130/	Standortfolge	TrackAggregator	/TC070/
/LF140/	Export	GpxXmlWriter, OptimizationPipeline	/TC100/
/LF150/	Export	SafeFileSink	Demo, SafeFileSinkJimfsTest.writesInsideBase
/LF160/	Export	GpxResult	/TC080/, /TC100/
/LF170/	Trackoptimierung	NthPointOptimizer	/TC110/
/LF180/	Trackoptimierung	MinDistanceOptimizer	implizit /TC110/, Demo
/LF190/	Trackoptimierung	LineCollinearityOptimizer	implizit /TC110/, Demo
/LF200/	Trackoptimierung	DouglasPeuckerOptimizer	implizit /TC110/, Demo
/LF210/	Externe Referenz	W3wHttpClient, NoopW3wClient	Demo (NoopW3wClient)
/LF220/	Externes Caching	W3wHttpClient	implizit /LF210/, Code-Re- view
/LF230/	Validierung	ValidationException	/TC020/
/LF240/	Validierung	ValidationException	/TC030/
/LF250/	Export	SafeFileSink	/TC090/

Tabelle 18: Vollständige Traceability-Matrix

4.2 Objektorientiertes Analyse- und Entwurfsmodell (OOA/OOD)

Dieser Abschnitt trennt bewusst zwei Sichten: Die objektorientierte Analyse (OOA) hält fest, *was* die Domäne ausmacht; der objektorientierte Entwurf (OOD) legt fest, *wie* sie technisch umgesetzt wird. Die Schichtenarchitektur steht in Kap. 2.1.

4.2.1 OOA: Domänenmodell

Die Analyse betrachtet die Fachlichkeit, bevor technische Entscheidungen einfließen. Sie hält fest, welche Konzepte die Domäne kennt und wie sie zusammenhängen, noch ohne konkrete Klassen, Schnittstellen oder Frameworks zu definieren. Die Assoziationen sind daher fachlich zu lesen, die Multiplizitäten bewusst pragmatisch gehalten.

Konzept	Beschreibung	Multiplizität
Peilung	Fachlicher Vorgang vom Start bis Complete/Abort.	1 pro Session
Zielpunkt	Geo-Koordinate (WGS84).	1
Standortfolge	Zeitlich geordnete GPS-Fixes.	0..*
Peilungsgrößen	Abgeleitete Werte (Azimut, Distanz, Ordinal).	0..1 vor erstem Fix
Track	Sammlung von Segmenten.	0..1
Export	GPX-Dokument als Artefakt.	0..1 nach Abschluss

Tabelle 19: Fachliches Domänenmodell (OOA): Konzepte und ihre Beziehungen.

Assoziationen: Peilung hat genau einen Zielpunkt und genau einen Track. Der Track enthält beliebig viele Trackpunkte. Die Peilung kann optional einen Export (GPX) erzeugen. Standortfolge: 0..*.

4.2.2 OOD: Technischer Feinentwurf

Der Entwurf überführt die fachlichen Konzepte in konkrete Klassen und Schnittstellen und ordnet ihnen ein Stereotyp zu, dazu zählen das Value Object, Domain Service, Infrastructure oder Port. Auffällig ist die Trennung über Strategie- und Port-Interfaces (`TrackOptimizer`, `W3wClientPort`, `ClockPort`): Sie hält die Domäne frei von technischen Abhängigkeiten und erlaubt, im Test echte Implementierungen durch Mocks zu ersetzen.

Klasse / Interface	Stereotyp	Kernverantwortung
BearingSession	Interface (API)	Öffentlicher Session-Vertrag.
DefaultBearingSession	Entity/Controller	Zustandsautomat IDLE/ACTIVE/COMPLETED/ABORTED.
SessionConfig	Value Object	Immutable Konfiguration; Builder-Pattern.

GeoCoordinate	Value Object	WGS84 lat/lon validiert.
GpsFix	Value Object	Zeit + Position + optional HDOP/Speed/Elevation.
BearingCalculator	Domain Service	Haversine-Distanz, Azimut, Ordinalrichtung.
TrackAggregator	Domain Service	Rohspeicher, Segmente, Punktbudget.
GpxXmlWriter	Infra-structure	GPX 1.1, XML-Escaping, UTF-8 (/LF140/).
SafeFileSink	Infra-structure	Atomares Dateischreiben, Path-Traversal-Schutz.
TrackOptimizer	Interface (Strategy)	Optimierungsstrategie austauschbar.
W3wClientPort	Interface (Port)	Reverse-Lookup-Abstraktion (/LF210/).
ClockPort	Interface (Port)	Testbarkeit via Clock-Injection (/LL070/).
BearingListener	Interface (Observer)	Ereignisbenachrichtigung für den Host.
NthPointOptimizer	Domain Service	Jeder n-te Punkt; Start/Ende erhalten (/LF170/).
MinDistanceOptimizer	Domain Service	Mindestabstand in Metern (/LF180/).
LineCollinearityOptimizer	Domain Service	Kollinearitätsreduktion (/LF190/).
DouglasPeuckerOptimizer	Domain Service	Douglas-Peucker mit metrischem Epsilon (/LF200/).

Tabelle 20: OOD-Klassenübersicht: Zentrale Entwurfsklassen und Schnittstellen.

Das Klassendiagramm visualisiert diese Entwurfsklassen mit ihren Stereotypen und Beziehungen. Gut erkennbar sind die Strategy-Hierarchie (`TrackOptimizer` mit vier Optimierern), der Observer (`BearingListener`) und die fünf Ports, die die Domain technologiefrei halten.

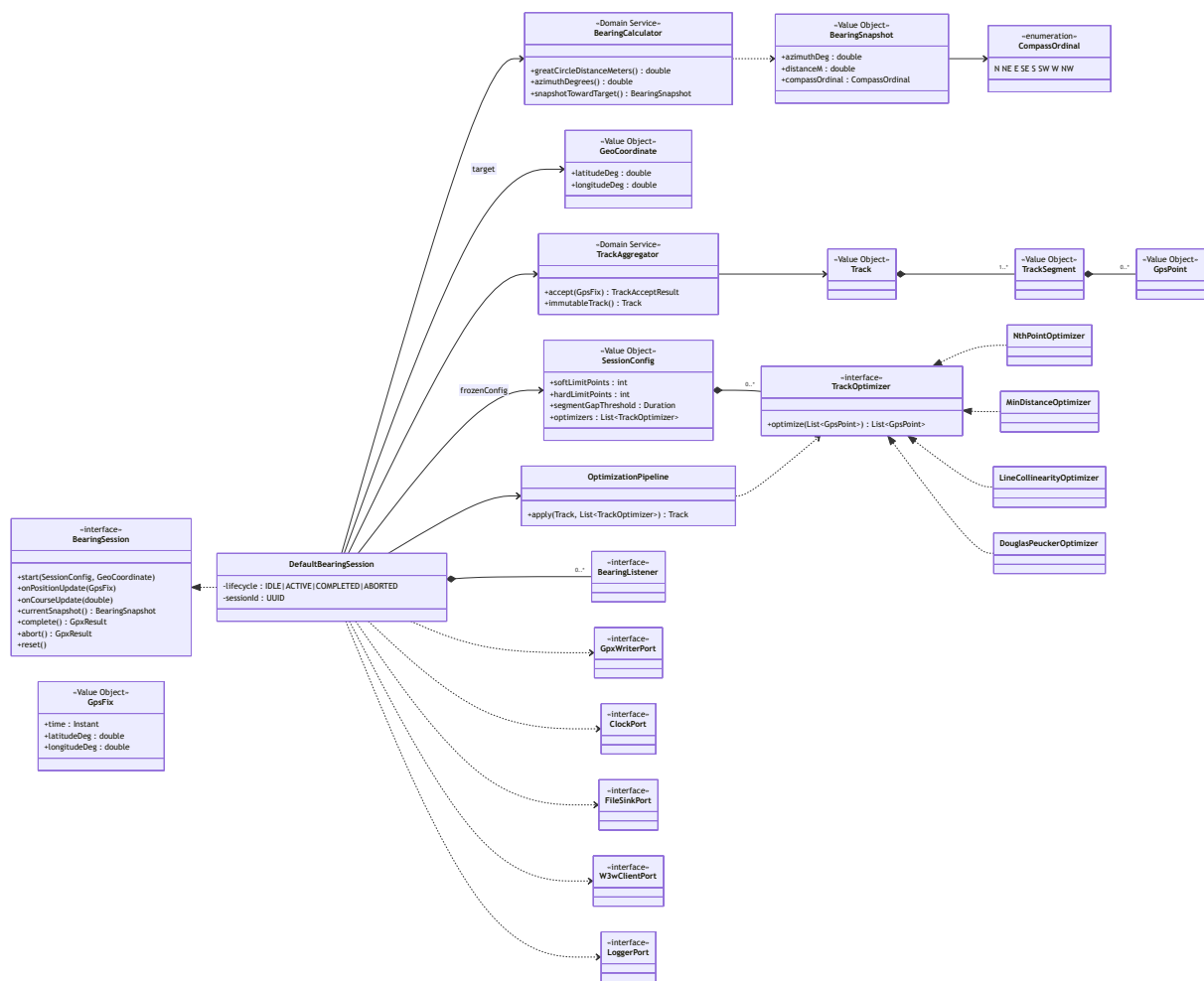


Abbildung 20: Klassendiagramm (OOD): Fassade, Value Objects, Domain Services, Strategy, Observer und Ports.

Das Objektdiagramm zeigt eine Momentaufnahme einer aktiven Session mit zwei gespeicherten Trackpunkten in einem Segment und konkretisiert damit das Klassenmodell.

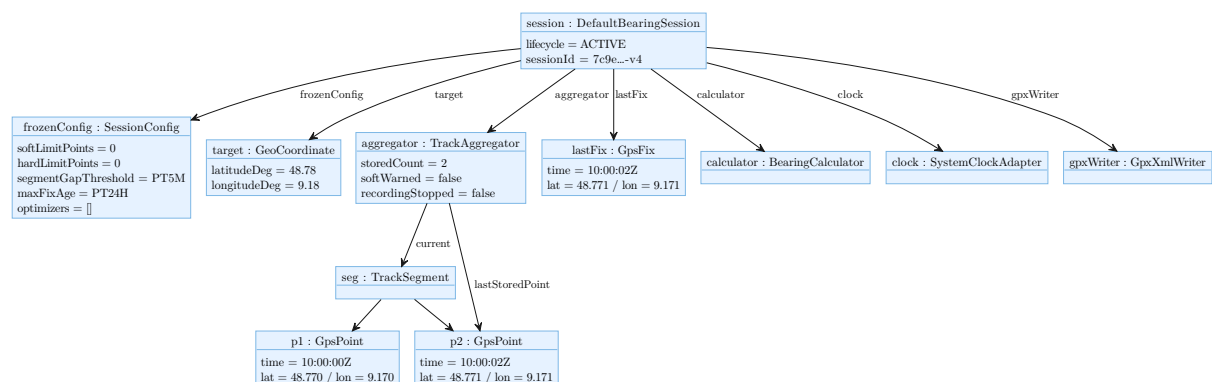


Abbildung 21: Objektdiagramm: Momentaufnahme einer aktiven Session (ACTIVE, zwei gespeicherte Punkte).

4.2.3 Zustandsdiagramm: Session-Lebenszyklus

Zustände: IDLE → ACTIVE (start) → COMPLETED (complete) oder ABORTED (abort). Rückkehr zu IDLE über reset() (/LF090/). Details zu Guards und Aktionen: Kap. 3.1 (/LF010/ ... /LF090/).

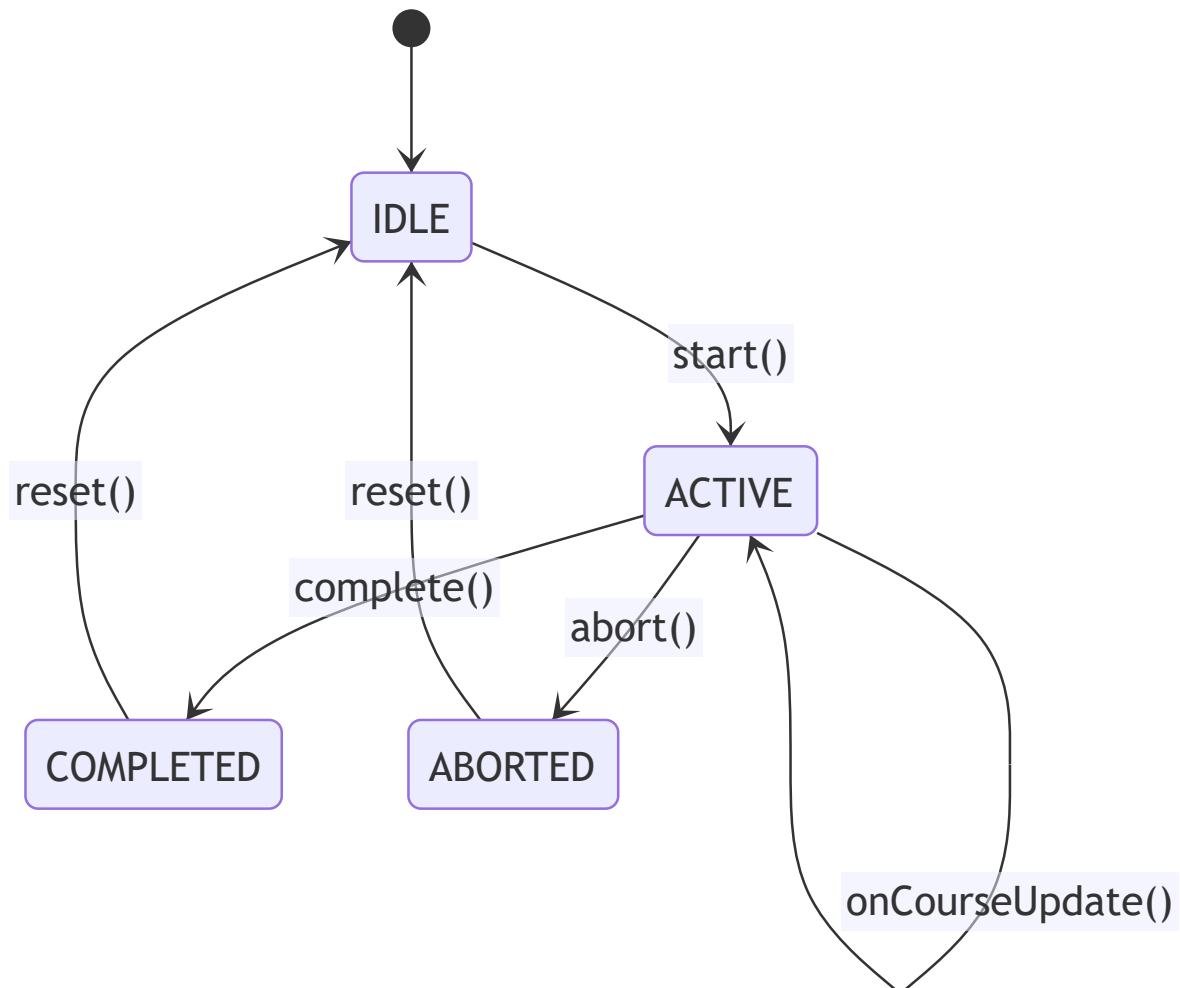


Abbildung 22: Zustandsdiagramm des Session-Lebenszyklus.

Die detaillierte Variante ergänzt die Übergänge um Bedingungen (in eckigen Klammern) und Effekte (nach dem Schrägstrich) und hält die Selbstübergänge im Zustand ACTIVE fest.

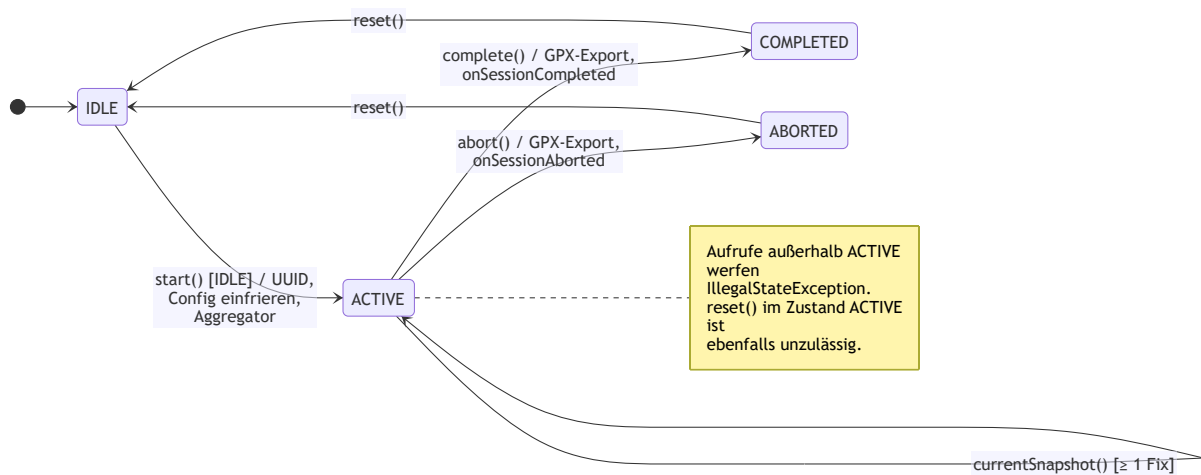


Abbildung 23: Zustandsdiagramm (detailliert) des Session-Lebenszyklus mit Guards und Effekten.

4.3 Struktur-, Architektur- und Verteilungssicht

Dieser Abschnitt ergänzt das Klassenmodell um die grobgranulare Bausteinsicht. Die Schichten- und Kommunikationsregeln stehen in Kap. 2.1.

Das Paketdiagramm zeigt die sieben Maven-Module und die enthaltenen Java-Pakete mit ihren erlaubten Abhängigkeitsrichtungen (Adapter → SPI, API → Domain/SPI/Adapter, Demo → API).

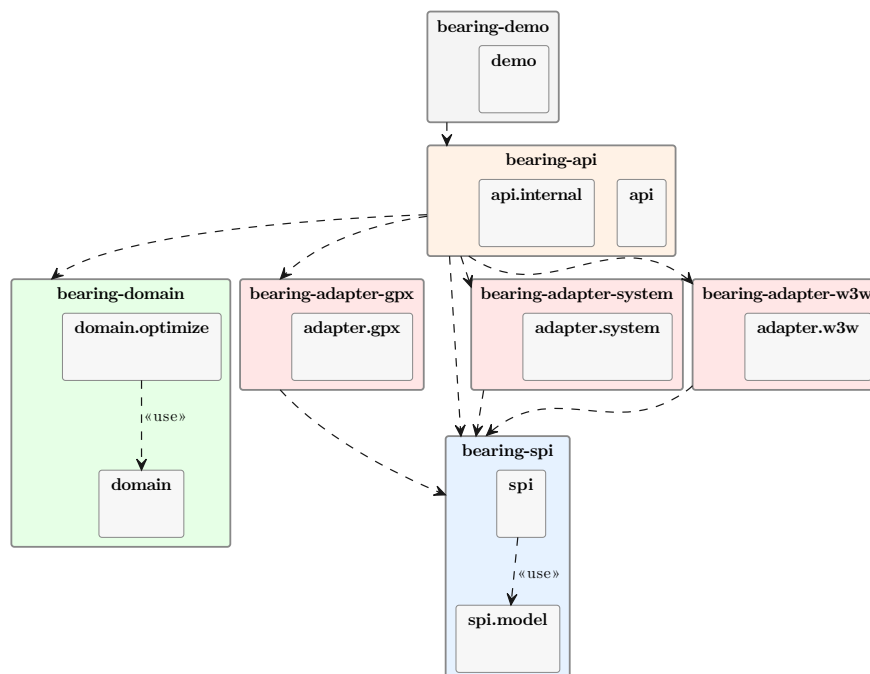
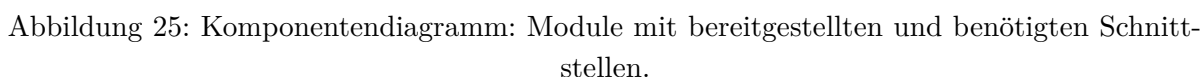


Abbildung 24: Paketdiagramm: Maven-Module und Java-Pakete mit ihren Abhängigkeiten.

Das Komponentendiagramm stellt dieselben Module als Komponenten mit angebotenen und benötigten Schnittstellen dar; die SPI-Ports entkoppeln die API von den Adaptern.



Connectors: aggregator liefert den Roh-Track, pipeline wendet die Optimierer an, calculator erzeugt Snapshots; alle Ports werden über den Konstruktor injiziert (BearingBootstrap).



Das Verteilungsdiagramm hält die physische Sicht fest: Die Bibliothek läuft als Artefakt im selben JVM-Prozess wie die Host-Anwendung; Dateisystem und What3Words sind optionale, abschaltbare Kanäle.

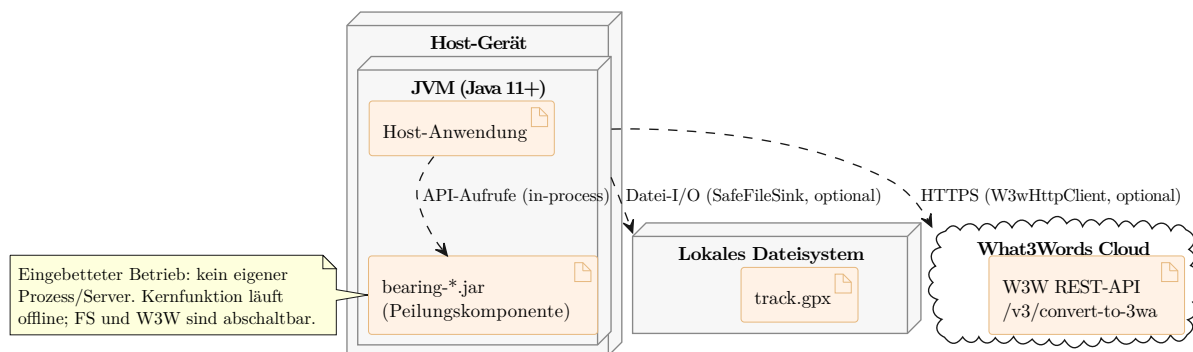


Abbildung 27: Verteilungsdiagramm: eingebetteter Betrieb mit optionalen externen Kanälen.

Über die Schichtenarchitektur (Kap. 2.1) hinaus nutzt der Entwurf zwei weitere Muster: Das Ports-and-Adapters-Muster (hexagonal) trennt den Anwendungskern über die SPI-Ports von den Adaptern, das Pipes-and-Filter-Muster beschreibt die Optimierer-Kette vor dem Export.

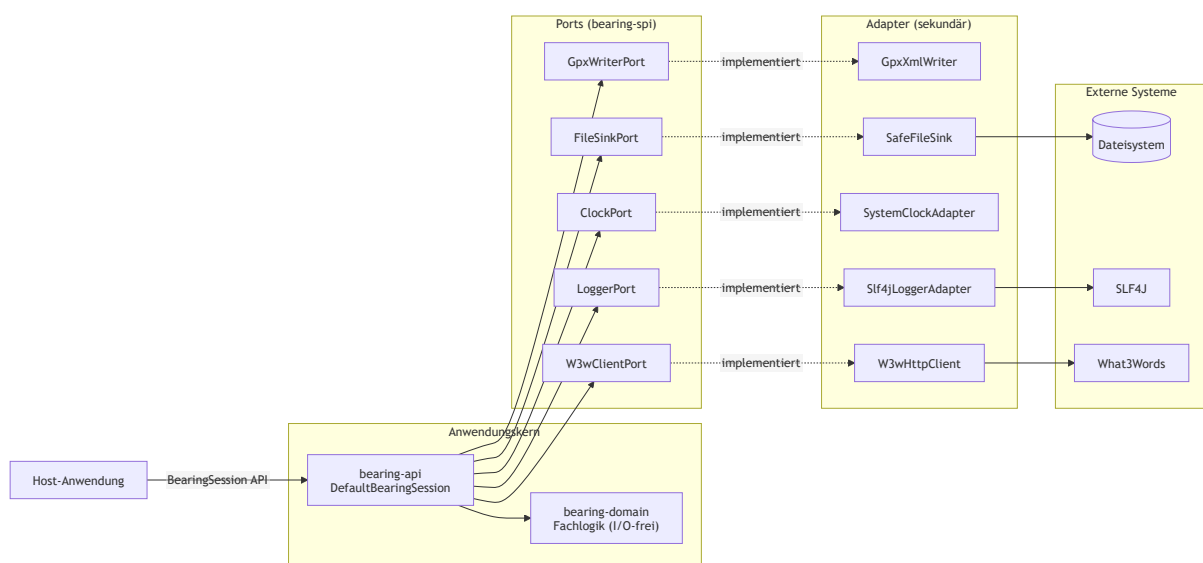


Abbildung 28: Architekturmuster Ports-and-Adapters (hexagonal).

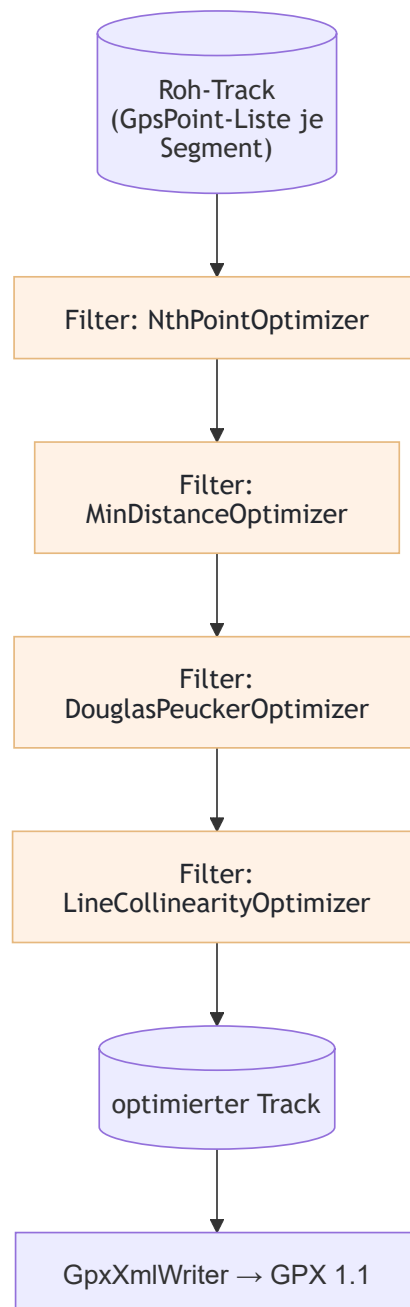


Abbildung 29: Architekturmuster Pipes-and-Filter: die Optimierer-Pipeline.

4.4 Dynamische Modelle und Strukturierte Analyse

Das Aktivitätsdiagramm fasst den gesamten Sessionverlauf von `start()` bis zum GPX-Export zusammen und verdichtet die anforderungsbezogenen Einzelaktivitäten aus Kap. 3.1.

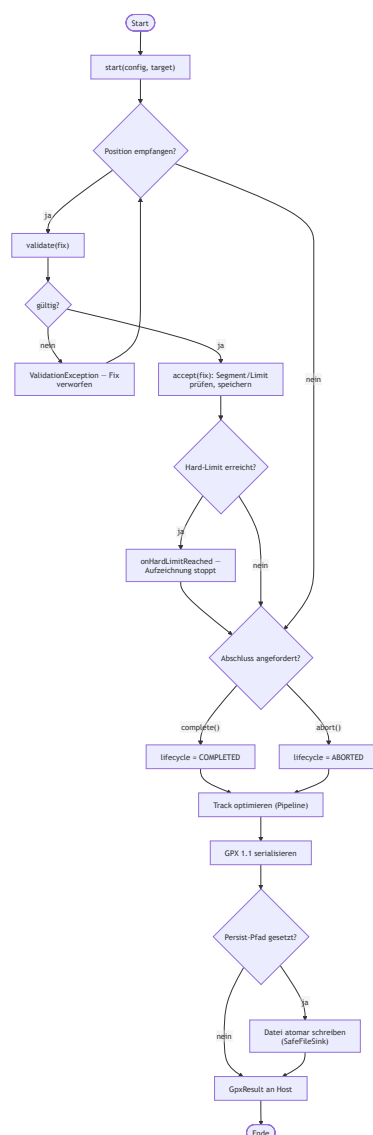


Abbildung 30: Aktivitätsdiagramm: vollständiger Verlauf einer Peilungssession.

Das Interaktionsübersichtsdiagramm ordnet die Sequenzdiagramme (Kap. 3.3) in den zeitlichen Gesamtablauf ein; die Doppelrahmen-Knoten verweisen auf je ein Sequenzdiagramm.

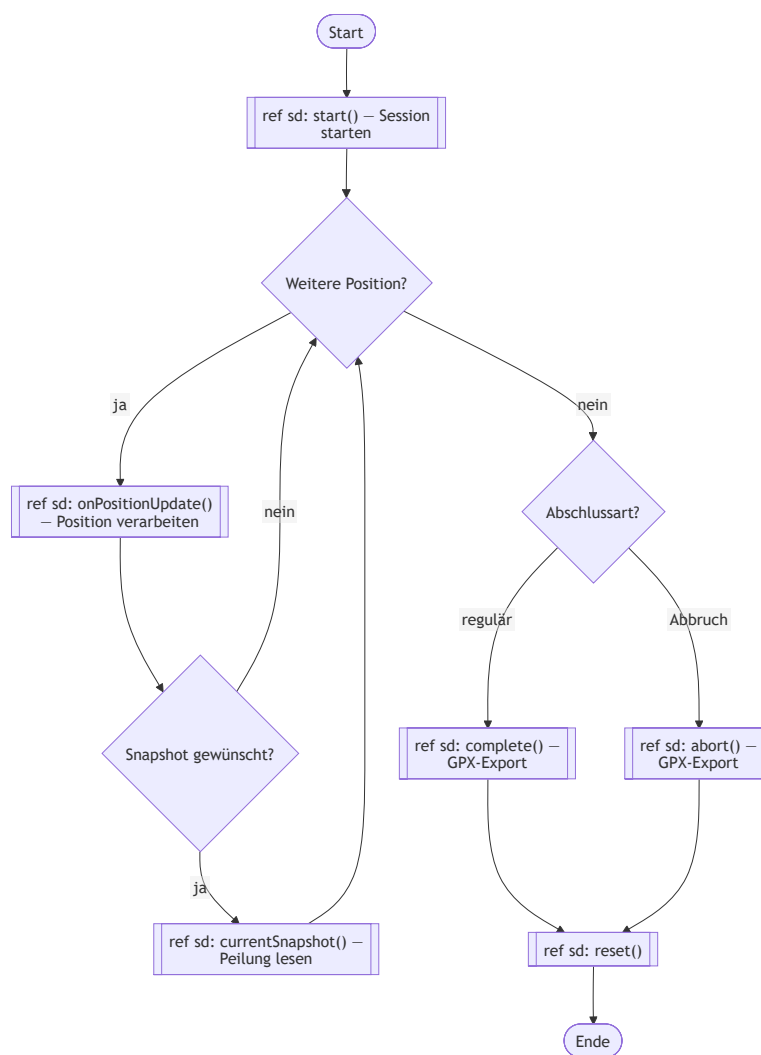


Abbildung 31: Interaktionsübersichtsdiagramm: Ablauf als Folge von Interaktionsverweisen.

Das Datenflussdiagramm der Stufe 1 verfeinert das Kontextdiagramm (Kap. 3.3) in sechs Teilprozesse mit den Datenspeichern Roh-Track und W3W-Cache.

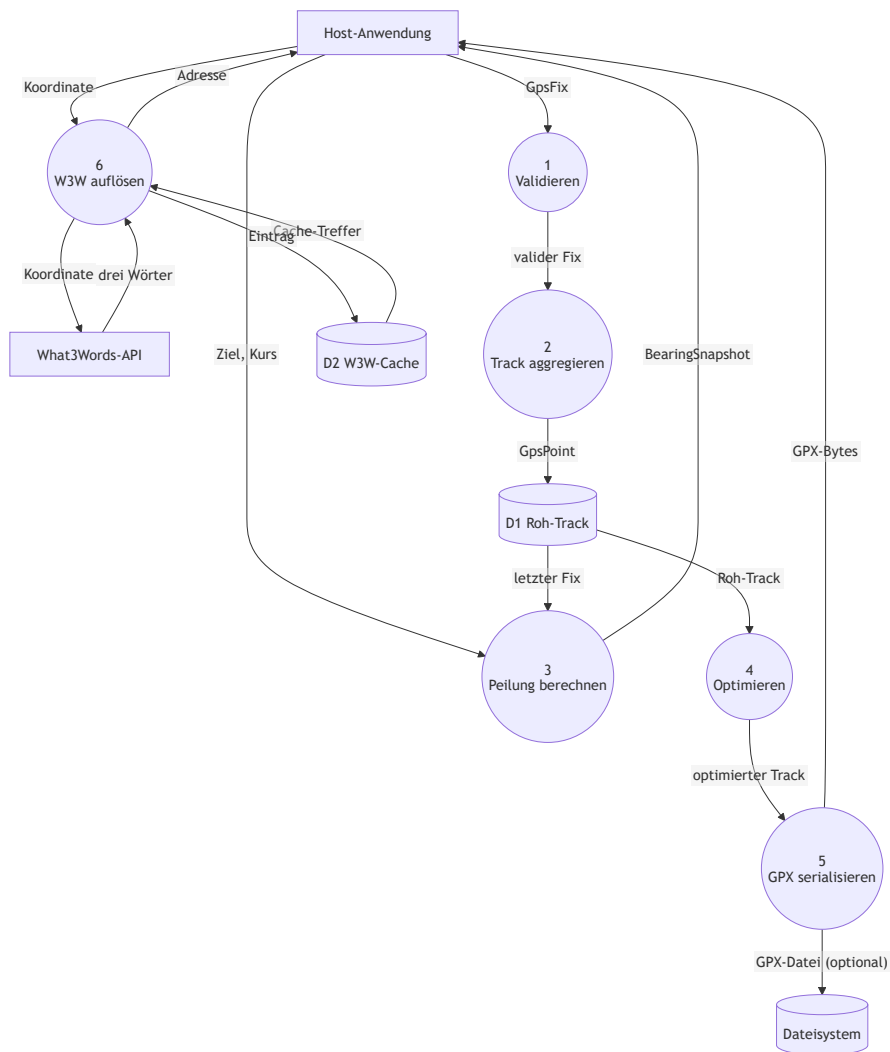


Abbildung 32: Datenflussdiagramm (DFD Stufe 1): Teilprozesse und Datenspeicher.

Das Petri-Netz liest den Session-Lebenszyklus als Stellen-Transitionen-Netz; die einzelne Marke macht die Invariante „genau eine aktive Session,“ anschaulich.

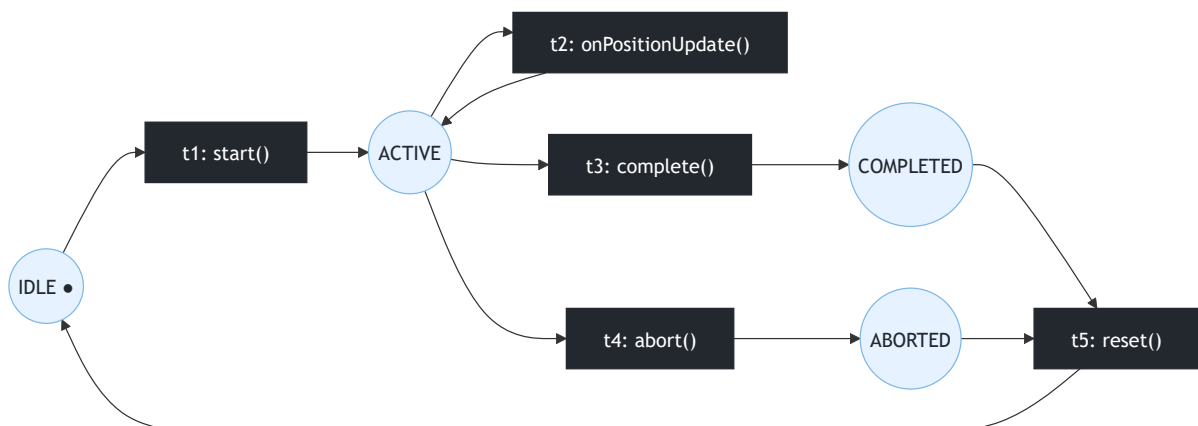


Abbildung 33: Petri-Netz des Session-Lebenszyklus (Stellen, Transitionen, eine Marke).

4.5 Haversine- und Azimutberechnung (Referenzalgorithmus)

Die folgende Beschreibung des Referenzalgorithmus dient der Übereinstimmung mit /LL020/ (Referenzabweichung $\leq \pm 1^\circ$). Die Implementierung nutzt `double` und muss Grenzfälle (identische Punkte, Polnähe) explizit absichern.

Eingabe: Breite/Länge von Standort φ_1, λ_1 und Ziel φ_2, λ_2 (in Radiant für die trigonometrischen Funktionen).

Haversine-Distanzberechnung:

$$\Delta\varphi = \varphi_2 - \varphi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1$$

$$a = \sin^2\left(\Delta\frac{\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\Delta\frac{\lambda}{2}\right)$$

$$d = 2R \cdot \arcsin(\sqrt{a})$$

wobei

$$R = 6.371.000 \text{ m}$$

der mittlere Erdradius ist.

Azimutberechnung:

$$\alpha = \text{atan2}(\sin(\Delta\lambda) \cdot \cos(\varphi_2), \cos(\varphi_1) \cdot \sin(\varphi_2) - \sin(\varphi_1) \cdot \cos(\varphi_2) \cdot \cos(\Delta\lambda))$$

Anschließend Normalisierung auf

$$[0^\circ, 360^\circ)$$

: azimuth = (alpha + 360°) mod 360°.

Grenzfälle:

- Identische Punkte ($d = 0$): Azimut undefiniert $\rightarrow$ Rückgabe 0° per Konvention.
- Polnähe: Numerische Instabilität durch

$$\cos(\varphi)$$

$\rightarrow$

$$0$$

muss durch Klammertests abgesichert werden.

- Antipodische Punkte ($d \approx \pi R$): Azimut mehrdeutig $\rightarrow$ dokumentieren und testen.

4.6 Literatur- und Normenverweise

Quelle	Kontext im Projekt
ISO/IEC/IEEE 29148:2018	Requirements Engineering, Traceability
ISO/IEC 25010:2011	Produktqualitätsmodell (Qualitätsmatrix Kap. 3.5)
IEEE 830-1998	Software Requirements Specifications (Dokumentaufbau)

GPX 1.1 Schema	XML-Export, Namespace http://www.topografix.com/GPX/1/1
WGS84 / EPSG:4326	Koordinatenreferenzsystem für alle Geo-Eingaben
What3words API Docs	Optionales Reverse-Lookup-Interface
OWASP	Path Traversal, XML-Sicherheitsrichtlinien
Gamma et al.: Design Patterns (GoF)	Strategy, Observer, Builder, Factory
Bloch: Effective Java	Immutability, Exceptions, API-Design
JUnit 5 User Guide	Teststrategie und Testframework
SLF4J Manual	Beobachtbarkeit und strukturiertes Logging
Douglas, Peucker (1973)	Algorithmus zur Linienvereinfachung /LF200/
SOPHIST-Methode	Anforderungsformulierung nach SOPHIST-Regeln
ISO 8601	Zeitstempel-Format UTC; <code>DateTimeFormatter.ISO_INSTANT</code>
Apache Maven Docs	Build-System, Profilverwaltung (w3w-Profil)

Tabelle 21: Primäre Normreferenzen und technische Quellen des Projekts.